密级:_____

中国科学院大学
University of Chinese Academy of Sciences

硕士学位论文

基于龙芯 3A 平台的 Android 4.0 操作系统移植与优化

作者姓名：孙浩志

指导教师：胡伟武　研究员

中国科学院计算技术研究所

学位类别：工学硕士

学科专业：计算机系统结构

研究所：中国科学院计算技术研究所

2013 年 5 月

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

Porting and Optimization of Android 4.0 Operating System based on Loongson-3A Platform

By

Sun Haozhi

A Thesis Submitted to

University of Chinese Academy of Sciences

In partial fulfillment of the requirement

For the degree of

Master of Engineering

Institute of Computing Technology, Chinese Academy of Sciences

May, 2013

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

声　　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书。）

作者签名：　　导师签名：　　日期：

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

摘 要

Android 操作系统是一款基于 Linux 内核的开源操作系统，目前在以手持移动设备为代表的各类硬件平台上均具有广阔的应用前景。龙芯 3A 是我国首款国产商用 4 核通用处理器，可应用于个人计算机、智能终端设备等多个领域。将 Android 操作系统移植到龙芯 3A 处理器平台，是推进国产处理器技术与 Android 系统软件技术相结合的基础性工作，对国产处理器应用领域的拓展具有重要意义；同时，对 Android 系统中的 Dalvik 虚拟机进行针对性优化，是进一步提升 Android 系统性能、增强 Android 软硬件产品竞争力与用户体验的关键。

本文以龙芯 3A780E 参考系统为目标平台，基于总结归纳大量系统移植类问题所得出的流程与方法，详细阐述了 Android 系统移植中 Android Linux 内核移植、文件系统核心模块移植及重构等关键问题的解决方案；并在此基础上对 Dalvik 虚拟机执行效率优化问题进行了深入研究，其中首先阐述了针对龙芯 3A 处理器的 Dalvik JIT 编译器移植方案；然后提出并实现了基于复合评价模型的代码热度检测算法，该算法可以进一步提升 Dalvik JIT 即时编译模式的执行效率，并且具有平台通用性可适用于龙芯平台以外的其他处理器平台。

经过实验评估，采用上述方案移植的 Android 4.0.4 操作系统已在目标平台上成功运行，并且具备较高的实用性；同时优化前后的性能测试对比结果表明，龙芯 3A-Android 系统在启用 Dalvik JIT 编译器后可获得 2 至 4 倍的性能加速比，采用基于复合评价模型的热度检测算法后可在此基础上进一步获得 5%至 8%的性能提升。

关键词：计算机软件；Android 系统移植；Dalvik 虚拟机；即时编译

中国科学院计算技术研究所 鄢玺 2025-03-20 16:12:07

ABSTRACT

Android is an open source Linux-based operating system widely used on mobile devices and other hardware platforms. Loongson-3A is a quad-core general-purpose CPU applied in personal computer and smart terminal devices. Porting Android to Loongson-3A platform is the foundation of integrating domestic CPU technology and Android software platform, and holds great significance to expand the application fields of Loongson CPU product. And the optimization of the Dalvik VM in Android is the key to improve Android system performance and user experience.

This thesis describes the key solution to port Android Linux kernel and Android file system to Loongson-3A platform, based on a methodology summarized from many operating system porting issues. On the basis of system porting, this thesis takes a further study on the Dalvik VM optimization, it first describes the solution to port Dalvik JIT compiler to Loongson-3A platform, and then proposes and implements a hot spot detector based on the Comprehensive Evaluation Model. This detector can improve the efficiency of JIT compilation, and it is able to be applied on other architecture beside Loongson with great generality.

The testing and evaluation in this thesis shows that Android 4.0.4 operating system ported following the solution above has been run successfully on target platform with high practicality. The result of comparative test indicates that Loongson3A-Android system can reach 2-4 times performance speedup after enable the ported JIT compiler, and the application of hot spot detector based on the Comprehensive Evaluation Model will result 5%-8% performance boost further.

Key Words: computer software, Android porting, Dalvik VM, Just-in-Time compilation

中国科学院计算技术研究所 鄢玺 2025-03-20 18:12:07

目 录

摘 要 I
目 录 III
图目录 V
表目录 VI
第一章 引言 1
1.1 课题背景 1
1.2 研究目标与意义 3
1.3 相关研究现状 4
1.4 本文的工作与结构 6
第二章 Android 系统架构及关键组件分析 9
2.1 Android 系统整体架构 9
2.2 Android 定制的 Linux 内核 11
2.3 Android 本地共享函数库 13
2.4 Dalvik 虚拟机 15
2.4.1 Dalvik 虚拟机的执行流程 16
2.4.2 Dalvik 虚拟机的架构特点 17
2.5 Android Java 核心库与应用程序框架 19
2.6 本章小结 20
第三章 基于龙芯 3A 平台的 Android 系统移植 21
3.1 目标平台硬件配置概述 21
3.2 Android Linux 内核移植 22
3.2.1 Android 定制驱动的移植 22
3.2.2 Android 定制驱动的 64 位兼容模式开发 23
3.3 针对龙芯 3A 的 Android 编译系统构建 27
3.4 Bionic 与 Dalvik 的移植 29
3.5 Android 图形子系统的重构 30
3.6 本章小结 33
第四章 移植成果的测试与评估 35
4.1 Android 文件系统的部署 35

4.2 系统可用性的测试及评估 36
4.3 移植系统的应用兼容性测试及分析 38
4.4 本章小结 39
第五章 基于龙芯 3A 平台的 Dalvik 虚拟机优化 41
5.1 Android 应用的性能模型构建及分析 41
5.2 Dalvik JIT 即时编译技术剖析 42
5.3 基于龙芯 3A 的 Dalvik JIT 编译器移植 46
5.4 基于复合评价模型的热度检测算法优化 48
5.5 优化效果的对比测试与评估 52
5.6 本章小结 53
第六章 总结与展望 55
参考文献 57
致 谢 i
作者简历 ii

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

图目录

图 1-1　龙芯 3A 处理器架构 2
图 1-2　MIPS 架构细分指令集兼容关系 5
图 2-1　Android 系统架构 9
图 2-2　Android 系统各层次依赖关系 10
图 2-3　基于 Dalvik 虚拟机的 Java 程序运行流程 16
图 2-4　jar 文件与 dex 文件的结构对比 17
图 2-5　Java 字节码与 Dalvik 字节码格式对比 18
图 3-1　系统移植类问题的一般解决方法 21
图 3-2　Linux 中 ioctl 调用的执行流程 24
图 3-3　compat_ioctl 接口与 ioctl 接口的实现对比 26
图 3-4　compat_ioctl 接口在驱动接口表中的添加 27
图 3-5　Android 源代码组织结构 27
图 3-6　Android 图形子系统 Server 端架构 31
图 3-7　龙芯 3A-Android 系统采用的图形子系统 Server 端架构 32
图 4-1　Android 文件系统组织结构 35
图 4-2　龙芯 3A-Android 系统基本功能测试场景 37
图 4-3　龙芯 3A-Android 系统 3D 图形性能测试场景 38
图 4-4　龙芯 3A-Android 系统浏览器测试场景 39
图 5-1　Dalvik JIT 编译模式工作流程 43
图 5-2　针对龙芯 3A 的寄存器分配模块架构 47
图 5-3　Dalvik 中的代码热度检测流程 48
图 5-4　基于复合评价模型的热度检测算法流程 50
图 5-5　基于复合评价模型的热度检测算法在 Dalvik 中的实现 51
图 5-6　两种优化方法归一化后的性能对比 53

表目录

表 1-1　Android 操作系统版本演进历程 4
表 3-1　数据类型在 32 位与 64 位编译下的内存占用对比 25
表 4-1　龙芯 3A-Android 系统 2D/3D 图形性能测试结果 37
表 4-2　龙芯 3A-Android 系统应用兼容性测试部分结果 38
表 5-1　优化方法一与解释执行模式的性能对比 52
表 5-2　优化方法二与优化方法一的性能对比 53

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

第一章 引言

1.1 课题背景

Android（中文译名安卓）是一套基于 Linux 内核构建的开源操作系统和软件平台[1]，其最早版本由一个名为开放手持设备联盟（OHA，Open Handset Alliance）的商业组织于 2007 年 11 月 5 日正式发布，该组织是由 Google 公司（Google Inc.，中文译名谷歌）领导数十家软件开发商、手机制造商和芯片制造商，以及电信运营商一同创立的[2]。在最初版本发布后不久，Google 公司即采取 Apache 免费开原许可证的授权方式，公开了 Android 操作系统的源代码，并在此基础上创建了一个旨在针对 Android 操作系统进行持续开发和改进的开源项目，即 Android 开放源代码项目（AOSP， Android Open Source Project）[3]。

Android 操作系统最初的设计目标，即在于为手持移动设备提供智能化的人机交互体验[4]。从其发布至今，得益于以 Google 团队为核心的软件开发者的持续改进，以及处理器、闪存等关键硬件部件的性能的不断提高，Android 操作系统的用户体验已经历经了数次显著的提升，因而其也越来越广泛地受到移动设备用户的认可。截至 2012 年 10 月，全球使用 Android 操作系统的设备数量已达 4.8 亿部，其设备类型已涵盖移动电话、平板电脑、智能电视以及智能机顶盒等多个消费类终端设备领域[5]。而如此强大的用户需求和广阔的市场前景，对于芯片制造、移动设备制造等上游企业也产生了巨大的影响。芯片制造企业中如高通、NVIDIA、英特尔等，移动设备制造企业中如三星电子、华为等，均是开放手持设备联盟的成员，以上硬件企业近年来均在 Android 设备的研发中投入了大量资源，并且持续进行针对自身硬件平台对 Android 系统进行优化和改造。

Android 操作系统之所以能够成功地迅速推广，良好的用户体验带来的产品竞争力可谓其外在的市场因素；各相关软硬件企业的投入与支持可谓其内在的商业因素；而除此之外另一关键因素即是其内在的技术因素——简而言之，Android 操作系统在架构设计方面采用了精妙的层次化结构，系统的不同层次之间实现了较为彻底的解耦。而基于此种设计思想，Android 操作系统选择了 Java 语言作为其应用程序的开发语言，并且通过系统中的 Dalvik 虚拟机（Dalvik VM），Java 核心库以及应用开发框架，为上层应用提供了“沙盒”模式的运行环境，从而使应用开发得以具备良好的安全性、跨平台性以及可重构性。上述系统特性，在目标硬件平台种类繁多、软件系统改进频繁的技术大背景下，极大地增强了 Android 生态环境适应力与竞争力。

Dalvik 虚拟机是一款专门针对手持移动设备平台而设计开发的 Java 虚拟机（JVM，Java Virtual Machine），Dan Bornstein 是该项目的创始开发者[6]。Android 操作系统的 Java 运行时环境就是基于 Dalvik 虚拟机来构建的。与目前由 Oracle 公司持有并开发的针对桌面和服务器平台的 Java 虚拟机——HotSpot 不同，Dalvik 虚拟机需要针对手持移动设备的 CPU 频率低、内存容量小以及电池容量有限等特性，因而在虚拟机架构、字节码格式等方面的设计实现都与前者有诸多不同之处。

目前在移动电话、平板电脑以及智能电视等消费类终端设备的硬件系统设计中，大量方案均采用了基于 ARM（Advanced RISC Machine）架构的处理器，其中一个重要原因是 ARM 架构作为 32 位 RISC（Reduced Instruction Set Computing，精简指令集）处理器阵营的代表，其设计实现更符合消费类终端设备低功耗、高性能功耗比的设计要求[7]；与此同时，Dalvik 虚拟机以及其他 Android 系统底层模块也在后续的开发中集成了诸多针对 ARM 架构的优化，这也进一步促使 Android 系统在消费类终端设备领域保持极高的市场占有率。

而在上述提到的 RISC 处理器阵营中，除 ARM 架构外，我国科研工作者研发的龙芯系列处理器亦是 RISC 体系结构的典型范例。在龙芯系列处理器中，龙芯 3A 是我国首款国产商用 4 核通用处理器，该处理器的内部架构如图 1.1 所示[8]。

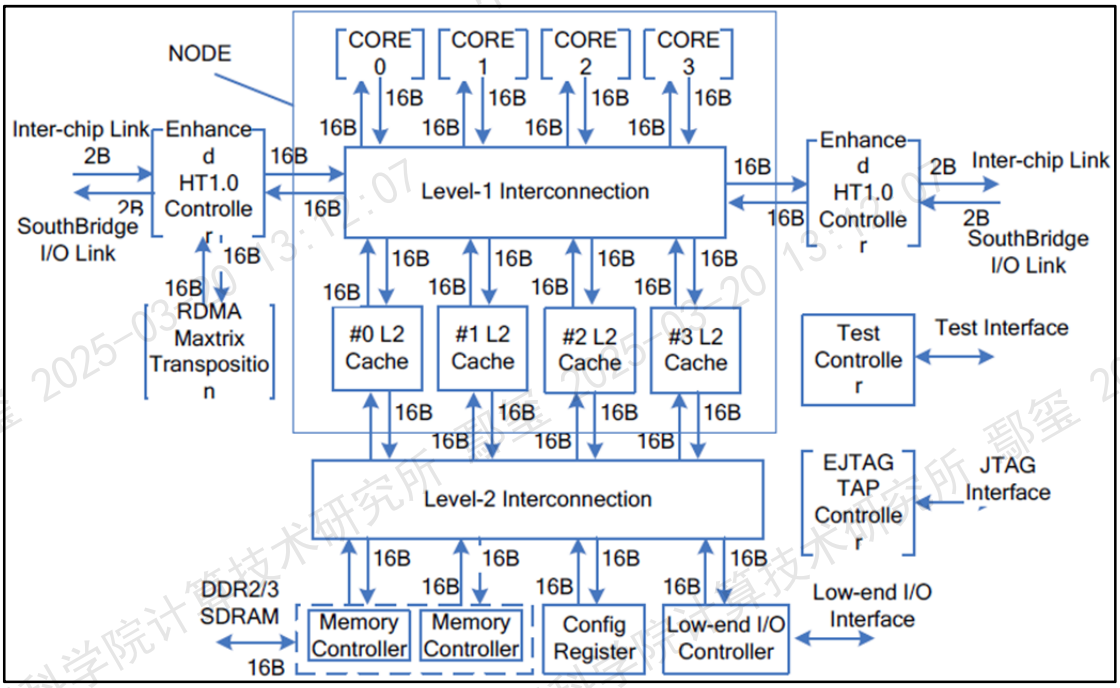

图 1-1 龙芯 3A 处理器架构

龙芯 3A 处理器采用 SMP 架构，片内集成 4 个 64 位 GS464 高性能处理器核，并兼容 MIPS64 指令集，每个核心内含有 2 个定点运算单元与 2 个浮点运算单元；处理器主频可达 1GHz 并支持动态降频，具有优异的性能功耗比[9]。根据其技术特性与市

场定位，龙芯 3A 处理器可以适用于个人计算机、智能终端设备等多个领域。

1.2 研究目标与意义

本文的研究内容及其具体目标由以下三个方面组成。

（一）基于龙芯 3A 平台的 Android 4.0 操作系统移植

本文首先以龙芯 3A780E 参考系统为目标平台，深入研究 Android 系统移植中的 Android Linux 内核移植、文件系统核心模块移植及重构等关键问题的解决方案。具体目标是在龙芯 3A780E 参考系统上成功运行 Android 4.0.4 操作系统，且系统能够按照 Android 操作系统所设计的交互模式进行正常操作。

（二）龙芯 3A-Android 系统的测试与评估

目前，对于龙芯平台运行 Android 操作系统的实际可用性和软件兼容性评估尚属空白。本文将采用工具测试结合手动测试的方法，对龙芯 3A-Android 系统的系统可用性以及应用兼容性进行测试评估。

（三）基于龙芯 3A 平台的 Android 系统性能优化

对基于龙芯 3A 平台的 Dalvik JIT 编译器移植以及 JIT 代码热度检测算法优化进行深入研究。通过上述两种优化方法的设计和实现，实现龙芯 3A-Android 系统的系统性能优化；并使用相关的系统性能测试工具，对优化效果进行量化评估和对比分析。

本文之所以选择上述研究内容作为研究工作的方向，是因为其在技术应用层面和行业发展层面均具有重要意义。

在技术应用层面，将 Android 操作系统移植到龙芯处理器平台，并构成具有实用价值的龙芯-Android 软硬件系统，是推进国产处理器技术与 Android 系统软件技术相结合的基础性工作，对国产处理器产品借助 Android 系统平台进一步拓展应用领域具有重要价值。同时，Android 系统中的 Dalvik 虚拟机优化是提升 Android 系统性能与 Android 产品用户体验的关键技术，对于进一步加强龙芯-Android 软硬件系统的整体竞争力具有重要意义。

而立足于行业发展的层面，对于有志埋首于核心电子器件、高端通用芯片以及基础软件行业进行创新与创业的企业来说，需要面对的不仅是与先发企业之间的技术竞争，还有更为艰难的、以生态环境对抗为核心的商业竞争。以传统个人计算机行业为例，原有的生态环境是由多家先发企业创立并主导的封闭生态圈，该生态圈的核心资源并不对后发的创业企业开放。在此种行业背景下，新兴企业或是游离于已被生态圈占据的广袤市场之外，难以寻求参与的机会；或是试图从零开始，打造一套由自己主导的新兴生态圈，但又由于基础差、起点低等问题，难以将新生态圈发展壮大。

相比之下，作为个人计算机行业的姊妹行业，消费类终端行业却展现出与前者明显不同的繁荣程度和创新能力。目前，该行业中仅 Android 平台的日均新增设备激活量即达到 150 万台，且硬件设备和应用程序都保持迅猛的增长态势。究其原因，这与 Android 生态环境的开放特性密不可分——Android 操作系统从发布伊始采取的免费和开源策略，奠定了这一新兴生态圈对待新兴参与者的天然的开放性；也正因此，Android 系统得以吸引整个产业链上下游的众多合作者，来共同创造和利用生态圈内巨大的市场资源。

对我国的终端设备企业和芯片企业而言，积极融入 Android 生态圈，将 Android 操作系统移植到自身平台，进而推出相应的系统级方案，才有机会利用 Android 生态圈的合作资源及用户群体，来拓展目标市场，提升自身产品的竞争力；而主动参加 Android 开源项目，不仅可以借鉴开源社区的成果提高自身的技术实力，更重要的是可以通过向开源社区回馈技术创新和优化经验等方式，丰富 Android 生态圈的多样性，成就企业和生态圈合作伙伴间的多赢。

1.3 相关研究现状

从首次发布至今，Android 操作系统的开发及优化一直是由 Android 开源项目（AOSP）在引领，在此其间已连续发布十余个 Android 系统的升级更新版本[10]，如表 1-1 所示。

表 1-1 Android 操作系统版本演进历程

版本号	版本名称	发布时间
4.2.x	Jelly Bean	2012 年 10 月
4.1.x	Jelly Bean	2012 年 6 月
4.0.x	Ice Cream Sandwich	2011 年 10 月
3.x.x	Honey Comb	2011 年 2 月
2.3.x	Ginger Bread	2010 年 12 月
2.2	Froyo	2010 年 5 月
2.1	Eclair	2009 年 10 月
1.6	Donut	2009 年 9 月
1.5	Cupcake	2009 年 4 月

截至本文开始写作时，Android 4.1.版本以下的各版本均已开放了源代码。在各个

版本的 Android 源码中，基于 ARM 平台的开发分支一直是 Android 系统开发的基准分支，任何与系统功能升级相关的源码修改都会首先在该分支上发布。

基于 x86 平台的开发分支从 Android 1.6 版本开始跟进该项目，即会周期性的将 ARM 基准分支的更新移植至 x86 平台，目前 x86 平台上已经移植完成并发布的最新版本为 Android 4.0.4。

基于 MIPS 平台的开发分支从 Android 2.2 版本开始跟进 ARM 基准分支的开发，截至本文写作时，MIPS 平台上已经移植完成并发布的最新版本为 Android 4.1.2。

Android 操作系统的源码开发共由 200 余个开源子项目组成。对于某一版本的 Android 源码而言，不同平台之间的源代码差异，主要集中在十余个与处理器架构以及硬件平台配置相关的子项目中，如 build、bionic、dalvik 等软件项目均在此之列。由此可见，将 Android 操作系统移植至新处理器平台的主要工作，即是根据目标平台的特性，对所有相关子项目中的有关源代码进行修改和重构。

目前已移植到 MIPS 平台的 Android 系统所支持的 MIPS 细分指令集包括 MIPS32、MIPS32r2，以及带有浮点部件的 MIPS32-fp，尚不支持 64 位的 MIPS64 指令集。自 MIPS-Android 4.0.4 版本之后，Dalvik 虚拟机中已经实现了针对标准的 MIPS32 处理器的汇编优化版字节码解释器，以及在虚拟机运行时可以编译生成 MIPS32 本地代码的 Dalvik JIT 编译器[11]。

除上述 Android 开源项目的相关工作外，也有研究者曾开展过将 Android 2.3 甚至更早期版本 Android 操作系统移植至龙芯处理器平台的相关工作。这些研究工作都选择龙芯系列的较早期产品——龙芯 2F 处理器为目标平台。如有研究者指出，龙芯 2F 处理器只兼容 MIPS III 指令集，MIPS-Android 中使用的 movn 等 MIPS32 指令在龙芯 2F 上并不支持[12]；所以需要对以上不兼容指令作等价替换。

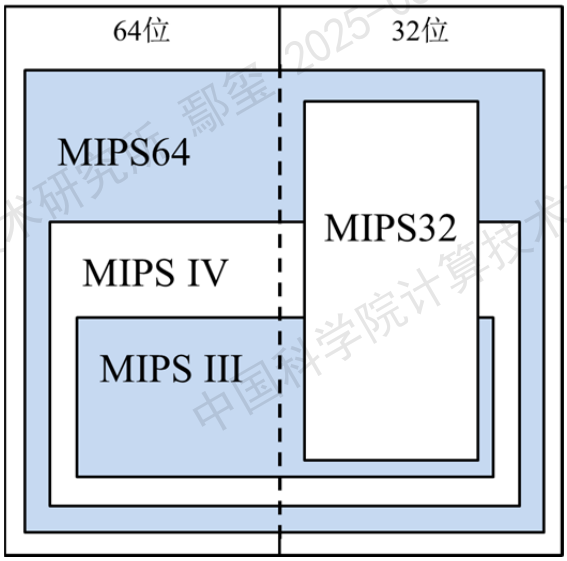

图 1-2 MIPS 架构细分指令集兼容关系

有研究者亦指出在 Android libAGL 本地图形库移植至龙芯 2F 平台的过程中，发现该库的实现中大量使用了龙芯 2F 不支持的乘加指令，因而采用 C 语言重构的方法对以上指令集兼容问题进行修正[12]。

由于本课题选择龙芯 3A 平台作为 Android 操作系统移植的目标平台，其研发时间晚于上述研究者使用的龙芯 2F 平台。且龙芯 3A 所采用的 GS464 核心已实现对 MIPS64 指令集的兼容，根据图 1-2 所示的 MIPS 架构细分指令集间的兼容关系可知，以上相关研究中涉及的 MIPS III 指令集兼容性问题对本课题不具有参考意义。

亦有研究者对 Android 图形子系统的移植以及硬件加速问题展开研究，其采用的主要方法是对 Android 硬件抽象层[13]以及显示缓冲区驱动[14]等系统软件模块进行针对目标平台的重构。由于上述方法具有平台相关性，因此无法在本课题所选目标平台上应用，参考意义有限。

而在基于龙芯平台的 Android 系统以及 Dalvik 虚拟机优化方面，目前已知的相关研究较少。如有研究者针对龙芯平台的过程调用特点，进行 Dalvik JNI 调用桥函数汇编重构等优化方法的相关研究[15]。而针对基于龙芯平台的 Dalvik JIT 编译器移植等问题，其研究现状尚属空白。

此外，由于龙芯 2F 处理器及其配套硬件设备的性能限制，上述相关研究中，均未对 Android 系统移植至龙芯平台后的实际可用性及应用兼容性进行评估，如对输入响应延迟、图像显示帧率等基本性能指标进行的量化描述。由此可见，以往研究在系统移植后的可用性和稳定性方面还存在较大的挖掘空间。

1.4 本文的工作与结构

本文的主要工作可以分为 Android 系统移植与系统性能优化两个方面。

在系统移植方面，本文完成了针对龙芯 3A 平台的 Android 4.0.4 操作系统源码重构与二次开发，成功实现了该系统在龙芯 3A 平台上的正常运行，并在此基础上完成了龙芯 3A-Android 系统的可用性与应用兼容性测试。

在系统性能优化方面，本文通过 Dalvik JIT 编译器的移植大幅度提高了龙芯 3A 平台上的 Dalvik 虚拟机性能；并且在此基础上提出并实现了适用于任意平台的基于复合评价模型的热度检测算法，采用该算法进一步优化了 Dalvik JIT 的执行效率。

以下为本文各部分内容的简要介绍。

第二章针对 Android 系统的层次化设计思想以及整体架构进行深入分析，其中将对 Android Linux 内核、Android 系统本地共享函数库、Davlik 虚拟机等关键系统组件的功能作用、关键技术以及平台相关性作具体剖析。

第三章首先对系统移植类问题的通用解决方法作一般性地归纳，在后续章节按照

该方法论展开本课题研究工作的阐述。第三章主要具体阐述本课题针对龙芯 3A 平台进行的 Android 系统的软件重构与二次开发工作，其中包括 Android Linux 内核的移植、针对龙芯 3A 的 Android 编译系统构建、Bionic 和 Dalvik 等软件模块的移植，以及针对龙芯 3A 平台的 Android 图形子系统重构等工作。

第四章起始将简要介绍 Android 文件系统目标程序在龙芯 3A 平台上的部署方法，然后具体列举龙芯 3A-Android 系统在可用性、基本性能以及应用程序兼容性等方面的测试数据，进而对测试结果进行分析评价。

第五章将首先通过构建和分析 Android 应用性能模型，简要解析系统优化切入点的选择依据，并对 Dalvik 虚拟机 JIT 即时编译技术的实现原理与工作机制进行分析；然后阐述基于龙芯 3A 的 JIT 编译器移植工作；此外本章还将重点介绍由本课题创新性提出并实现的 Dalvik JIT 优化方法，即适用于任意平台的基于复合评价模型的热度检测算法；最后将列举以上两种优化方法在相关性能测试集下的效果表现与测试结果。

第六章将对本文工作中所涉及的关键技术以及具有创新性的方法作简单总结，同时对未来的研究前景作初步展望。

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

第二章　Android 系统架构及关键组件分析

本章将对 Android 系统的系统架构、系统关键组件的功能，以及其目标平台相关性进行针对性地分析，为具体的系统移植工作提供理论指导。

2.1　Android 系统整体架构

任何操作系统都构建于符合一定要求的硬件平台之上。Android 系统对硬件性能的要求并不高，以 ARM 平台为例，其最低硬件配置要求为：处理器主频 600MHz，内存空间 128MB，外存储空间 1GB。

在硬件平台之上，即是基于 Linux 内核构建的 Android 操作系统。如图 2-1 所示是 Android 系统的整体架构，可以看出 Android 采用了优雅的层次化架构设计，从系统底层至顶层依次是 Linux 内核、本地共享库、用户态运行时环境、应用程序框架以及应用软件层；而 Android 文件系统即由除 Linux 内核以外的四个层次构成。

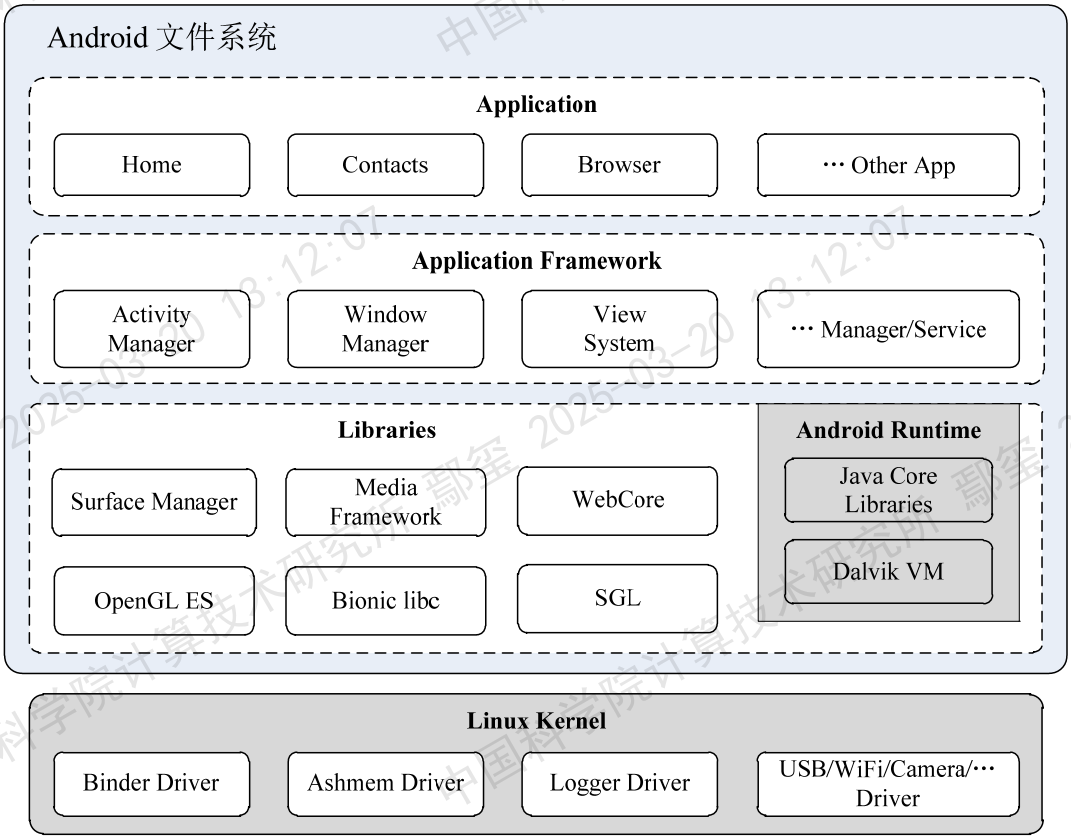

图 2-1 Android 系统架构

图 2-2 展示了 Android 系统中各层程序之间的依赖关系。Android 平台上应用程序

采用 Java 作为开发语言，基于应用程序框架层提供的一系列 Java API（Application Programming Interface）以及系统服务进行应用逻辑的开发；应用层与应用程序框架层的 Java 程序基于以 Dalvik 虚拟机为核心的 Java 运行时环境来运行；同时在 Java 程序中还可以使用 native 方法，即通过 Dalvik 虚拟机提供的 JNI（Java Native Interface）机制实现对本地代码的调用；最终，Dalvik 虚拟机以及采用 C/C++语言实现的本地共享库则都运行在底层的 Linux 环境之上。

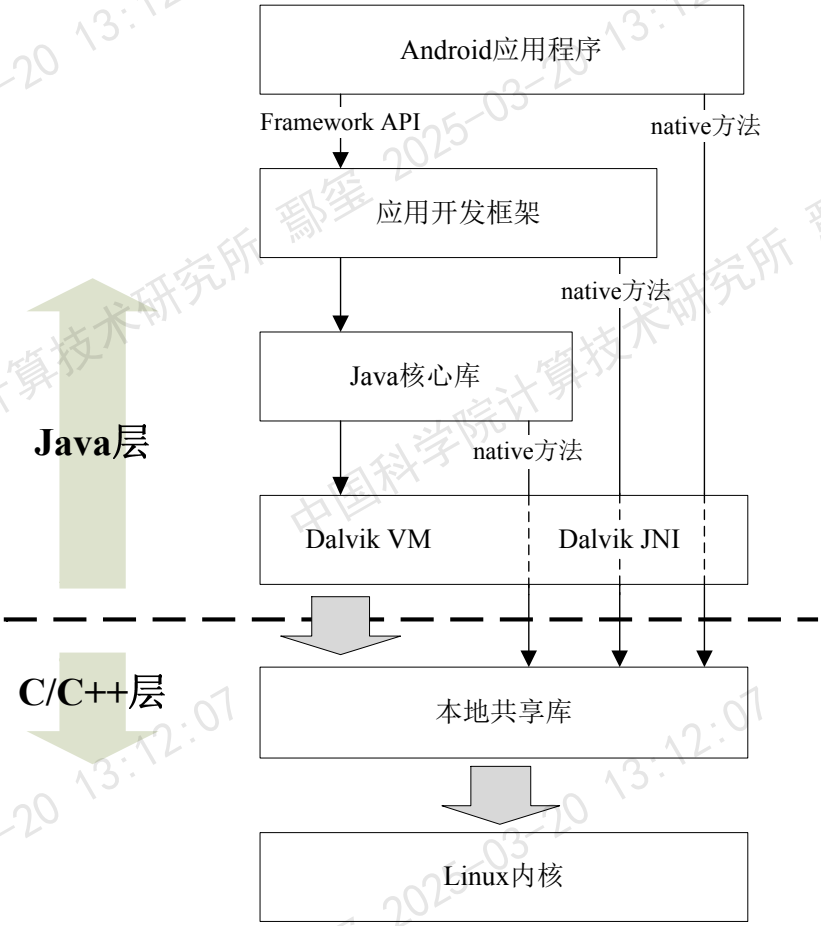

图 2-2 Android 系统各层次依赖关系

从上述应用开发以及运行依赖关系的分析中，可以明显看出 Android 系统采用层次化架构设计的优点：即每一层在对其上层提供统一服务的同时，可以屏蔽更低层次的实现细节，使得层与层之间耦合度降低，从而便于各个系统模块进行独立开发、移植以及调试[16]。

因此，Android 应用开发者一般只需要关注应用层本身以及应用程序框架层；而 Android 系统的移植者则需要关注 Android Linux 内核、本地共享库以及 Android 运行时环境。下面就分别这几层的功能、实现以及可移植性作深入分析。

2.2 Android 定制的 Linux 内核

Android 在开发之初就选择了 Linux 2.6 作为操作系统的内核，Android 系统升级至 4.0 版本之后也将所采用的 Linux 内核版本提升至了 3.x。Linux 内核所起的作用是在底层硬件与上层系统核心库之间构建了第一层次的硬件抽象，即对上层 Android 系统提供了内存管理、进程管理、基于用户权限的安全性管理以及底层硬件驱动模型等核心服务，同时屏蔽了大部分底层硬件实现的具体差异。

虽然同样都是基于 Linux 内核构建，但 Android 系统与其他基于 Linux 内核的操作系统相比还是有诸多不同之处。首先，Android 系统没有沿用在其他 Linux 操作系统上广泛使用的 glibc 本地共享库以及 Linux 平台的标准工具链，包括图形窗口系统也没有采用较为成熟的 X11 架构；这就导致传统的基于 glibc 库或 X11 架构编写的程序与 Android 系统并不兼容。

除了对以上 Linux 平台常用组件做了大量剪裁之外，Android 还在 Linux 内核中添加了若干新的模块，用以优化 Linux 内核在某些方面的性能，使其更适应 Android 系统的应用场景。目前 Android 文件系统的源码中并不包括 Android 定制的 Linux 内核，该内核是作为 Linux 内核的一个分支独立进行项目开发的。经过对其源码实现的分析，本文总结归纳出这些模块向 Android 文件系统提供的关键功能如下。

（1）Binder 驱动

Binder 是 Android 特有的一种进程间通信机制，该机制为运行在单机系统上不同进程，提供了类似于分布式系统中远程过程调用（RPC，Remote Procedure Call）的功能[17]。

在 Android 系统中，多媒体播放、视频音频信息获取以及各种传感器信号捕捉等功能都是作为服务由专门的 Server 程序负责管理，应用程序开发者只需要按照协议实现 Client，并与 Server 进行进程通信就可以获取这些服务，无需重新进行开发。目前 Linux 内核所支持的进程间通信机制包括管道、消息队列、共享内存、信号量以及 Socket，这几种已有机制并不适用于在系统资源量较紧张的手持移动设备上构建高效的 Client-Server 通信架构。鉴于此，更适合 Android 应用场景的 Binder 机制应运而生。

完整的 Binder 机制需要由 Client、Server、Service Manager 和 Binder 驱动程序这四个组件联合实现，其中 Client、Server、Service Manager 这三个组件都为用户态程序，在 Android 应用层和应用程序框架中实现；而 Binder 驱动程序则作为 Linux 内核模块，实现进程间通信的底层核心功能，即 Client 和 Server 可以依靠 Binder 驱动和 Service Manager 提供的基础架构，进行彼此间的通信。

Linux 内核在加载 Binder 驱动时会创建一个“/dev/binder”虚拟设备文件，该文

件对外提供 open，mmap，poll，ioctl 等标准文件操作接口，Client 和 Server 通过以上接口直接与 Binder 驱动程序进行通信；驱动程序根据其内部维护的客户端以及连接信息，在 Client 和 Sever 间传递通信数据包，从而使 C/S 间可以实现间接通信。

除了维护连接关系的核心功能外，Binder 驱动也是连接相关的接收数据缓冲区的实际管理者。在通信时，Binder 驱动会从发送进程的发送缓冲中将数据拷贝至由自己维护的接收缓冲区，并通过 mmap 只读映射的方式将接收缓冲映射至接收进程的进程空间。由此，只需通过 1 次内存拷贝即可实现发送进程与接收进程的数据传递。

（2）Ashmem 驱动

Android Linux 内核中的 Ashmem（Anonymous Shared Memory）驱动实现了一套匿名共享内存机制，该驱动辅助内核中原有的内存管理模块和共享内存机制，提供了进程间内存共享与有效内存管理这两个功能。

与 Binder 驱动一样，内核加载 Ashmem 驱动模块时，会创建一个“/dev/ashmem”虚拟设备文件，该设备文件对外提供了 open、mmap、release 以及 ioctl 四个文件操作接口。在 Android 系统的 C/C++层和 JAVA 层中都会对以上接口进行进一步封装，用户态程序可以通过这四个接口创建匿名共享内存、把共享内存映射至自身的进程空间并将其锁定；在共享内存使用完毕后将其解锁并再度交由 Ashmem 驱动进行内存块的管理与回收工作[17]。

当多个进程间需要进行匿名内存共享时，首先会由某个发起进程 A 创建匿名共享文件并获得其文件描述符，然后调用 mmap 接口将该文件映射至本进程的地址空间，最后通过之前已经分析过的 Binder 进程间通信机制，将该文件描述符告知另一目标进程 B；在此期间 Binder 驱动还会负责将进程 A 的文件描述符转换为进程 B 的文件描述符。如此目标进程 B 在获得该经转换的文件描述符之后，即可与发起进程 A 共享同一块物理内存区域。匿名共享文件所对应的物理内存，始终由 Ashmem 驱动中实现的相关算法负责管理与回收，因而保证了内存使用的效率与安全性。

（3）Logger 驱动

Android 系统自身实现了一个轻量级的日志记录系统，该日志系统的核心是 Android 为 Linux 内核添加的 Logger 驱动；而在上层，Android 系统则为用户空间提供了调用日志系统的 Java 接口和 C/C++接口，以及一个名为 logcat 的二进制工具。因此无论是工作在 Java 层的应用开发者，还是工作在 C/C++层的 Android 系统组件开发者，都可以利用这套系统定义并记录程序的运行信息，为开发调试提供支持[17]。

Logger 驱动在加载后会创建三个名为/dev/log/main、/dev/log/events 以及/dev/log/radio 的虚拟设备文件，提供相应的文件操作接口。Logger 驱动内部使用等待队列的结构来串行化发起日志读取的进程，并使用环形缓冲区来暂存日志数据，由于环形缓冲区中的节点是以单条日志为单位配备相关信息的存储结构，所以开发者在读

写日志时都是以条为单位进行操作，这大大提高了该套日志系统的易用性。

（4）Low Memory Killer 机制

Android 系统中的进程的生命周期全权由系统控制，即使用户手动关闭程序，其进程的上下文依然会在内存中保存一定的时间，如此设计的目的是追求程序的快速再次启动，为用户提供更佳的交互响应；然而这一机制会导致随着系统运行时间的推移物理内存被快速消耗。因此 Android 以 Linux 内核原有的内存耗尽释放机制（OOM，Out Of Memory killer）为基础，增加了定时检查、杀死冷进程，及时释放内存的 Low Memory Killer 辅助机制。

Linux 原有的 OOM 机制会在监测到系统可用内存不足时启动，通过一系列较为复杂的评分算法对所有存在的进程进行打分，得分高者将被判定为 bad 进程，进而被杀死以释放内存。而 Low memory killer 机制的启动机制为定时启动，在启动后 Low memory killer 会根据系统当前的剩余内存空间计算出一个警戒级数。根据 Linux 内核已有的进程管理机制，内核会在每个进程启动后为其设置了一个 oom_adj 值，可以简单认为该值与进程的重要性反相关，即 oom_adj 数值越高表示进程的重要程度越低；当警戒级数达到预设阈值后，Low Memory Killer 将检查所有 oom_adj 值大于警戒级数的进程，并把其中重要程度最低者或消耗内存最多者杀死。这套机制相比于原有的 OOM 机制更为简单高效，适于频繁定时地执行[18]。

除了以上介绍的若干较为重要的定制模块外，Android 系统还在 Linux 内核中添加了专用的电源管理策略，YAFFS2 文件系统支持，以及某些专用的输入输出设备驱动等多个新增功能，由于与本文的研究方向没有交集在此就不逐一列举分析了。

经过以上调研分析可以发现，全套 Android Linux 内核的移植是 Android 系统移植的基础，即首先应当实现在目标平台上可以稳定运行 Android Linux 内核。由于 Android 对 Linux 内核的修改都是以独立内核模块的形式实现的，因此在 Android Linux 内核移植时有两条不同的技术路线可选择：一是以目标平台上已有的 Linux 内核为主干，将 Android 对标准 Linux 内核的修改检出并合并至主干；二是以某一平台，如 ARM 或 MIPS 平台上已稳定的 Android 定制内核为主干，将目标平台的平台及体系结构相关修改添加至主干。在第三章中将针对本课题的实验环境对以上两条技术路线的利弊进行详细分析。

2.3 Android 本地共享函数库

在 Android 系统中，基于 Linux 内核层之上构建的是系统中的本地共享函数库层，共享函数库为应用层的 Java 程序乃至 Java 运行时环境中的 Dalvik 虚拟机提供最基础的运行本地运行环境以及本地函数调用。

Android 本地共享库的生成、部署以及使用方式沿袭了 Linux 平台上已有的传统：即在 Android 源代码中，共享库的组成函数均使用 C/C++语言开发实现；在目标平台本地，经编译后的共享函数库以一组.so 动态库文件的形式存储在文件系统的相关目录下。目标平台上其他由 C/C++语言开发的程序需要在开发编译时依照约定指明共享库名称，即可在运行时动态链接共享库中的函数；而 Java 层中程序可以通过 JNI 机制调用本地库函数。因此一般也可以将共享函数库层称作 C/C++层，与上层基于 Java 语言的层次相区别。

下面从实现功能的角度出发，详细列举分析 Android 系统中部分较为重要的本地共享库。

（1）Bionic libc

Bionic libc 是 Android 系统使用的标准 C 函数库，Android 用 Bionic 取代了其他 Linux 平台上常用的 glibc 库，但二者提供的函数功能基本相同。Bionic 相比于 glibc 主要有如下优点[16]：

A. 由于功能精简，Bionic 库文件的体积只有约 200KB，比 glibc 库文件减小了近 50%，其函数实现比 glibc 更为简洁，运行效率也更高；
B. Bionic 中实现了一个量级更轻，速度更快的 pthread 库，更适于移动设备处理器主频较低的应用场景；
C. Bionic 提供了一些 Android 系统特有的重要函数，如系统属性变量设置与获取函数 setprop/getprop，程序执行日志输出函数等。

（2）OpenGL ES

Android 系统中的图形窗口系统由 Java 应用层、本地共享库层、以及硬件驱动层中的多个组件共同组成，其中后端的图形引擎是基于 OpenGL ES 接口以及 EGL 接口来构建的。本地共享库中的图形库即负责封装实现标准的 OpenGL ES 和 EGL 接口，但从本地图形库以下至最底层的图形处理硬件之间的软件框架目前尚无法统一，因此在进行 Android 图形系统的开发和移植时，往往需要根据目标平台图形硬件的驱动支持情况来定制解决方案。

（3）Media Framework

早期的 Android 系统采用 OpenCore 多媒体框架来构建系统的多媒体核心库，从 2.3 版本之后开始改用 StageFright 框架，并逐步废止了 OpenCore 的使用。以上两种框架虽然在实现上有所差异，但对外均主要提供两方面的功能——第一是多媒体播放功能，即各种格式的音视频流的解码与回放；第二是媒体流记录功能，即音视频流以及图像的捕获与编码。

（4）Surface Manager

Android 系统中负责管理应用程序的图形绘制操作、合成 2D/3D 绘图流以及管理绘图缓冲区的库。基于该库中实现的管理结构及管理机制，图形显示缓冲区被抽象为多个逻辑 surface 供上层应用使用，应用程序只需要负责在自己分配的 surface 上进行绘图操作，多个 surface 之间的切换以及合成等功能则在该库中实现。

（5）HAL（Hardware Abstraction Layer）

Android 系统的本地共享库中包含了 HAL 硬件抽象层的实现。实现该层的目的是对 Linux 驱动进行高一级封装，即在内核空间驱动程序的基础上，在 Android 用户空间中实现了一系列统一程度、抽象程度更高的硬件操作接口。基于 HAL 层接口开发的用户空间程序既可以充分地使用移动网络、照相机、GPS 等硬件的功能，同时又无需关注硬件驱动的差异，保持极高的可移植性。此外，由于 HAL 层中的驱动程序不遵循 Linux 的 GPL 协议，因此硬件厂商可以在不公开源代码的情况下参与 HAL 层驱动的开发，这使得 HAL 层可以集成大量受知识产权制约的高质量驱动，对充分释放 Android 平台的硬件性能起到了很重要的作用。

除上述几个共享库之外，Android 系统还提供了十余个面向各类功能的共享函数库，其中大部分库函数具有平台无关性，因此在移植时只需要针对目标平台重新进行编译和部署即可，不需要对函数的 C/C++源码进行修改。但对于 Bionic libc 库、图形库、多媒体编解码库等与硬件驱动以及硬件设备耦合较为紧密的本地库，则需要从库函数源码甚至整个本地库架构入手，针对目标平台的具体情况进行修改。

2.4 Dalvik 虚拟机

Android 系统的应用层和应用程序开发框架层都采用 Java 作为开发语言。众所周知，Java 语言是一种解释型语言，使用 Java 语言编写的源代码在经过 Java 编译器编译后，生成由字节码（Bytecode）组成 class 文件；在目标平台上运行时，类加载器会将这些 class 文件加载到 Java 虚拟机的相应内存中，由虚拟机进行解释执行。由于单纯的解释执行是一种效率较低的运行方式，所以目前主流的 Java 虚拟机，如 Hotspot、IBM JVM 之中，除了含有最原始的解释执行模块，还包含有支持在运行时将 Java 字节码编译为目标平台本地机器码的 JIT（Just-In-Time）即时编译模块，虚拟机在实际运行时会根据某些条件自行选择采用解释执行或编译执行，而采用 JIT 技术后可以较大幅度地提高 Java 程序整体的运行效率。

Android 系统没有采用已有的较为成熟的 Java 虚拟机方案，而是针对手持移动设备 CPU 频率低、内存容量小以及电池容量有限等特性专门开发了 Dalvik 虚拟机，在 Dalvik 虚拟机上运行 Java 程序的流程与传统的 Java 虚拟机基本一致，即也分为编译、加载、解释执行或编译执行这几个环节。但是在各环节的运行细节以及虚拟机内

部架构的设计上 Dalvik 虚拟机有诸多独到之处。

2.4.1 Dalvik 虚拟机的执行流程

图 2-3 展示了在 Dalvik 虚拟机上运行 Java 程序的详细流程：java 源文件经过编译器 javac 的编译，生成 class 或 jar 文件；class 文件和 jar 文件可以直接输入到传统的 JVM 运行，但由于 Dalvik 虚拟机的输入文件格式为特有的 dex 格式，所以在运行前还需要使用 Dalvik 附带的 dx 二进制工具，将 class 文件转换为 dex 格式。

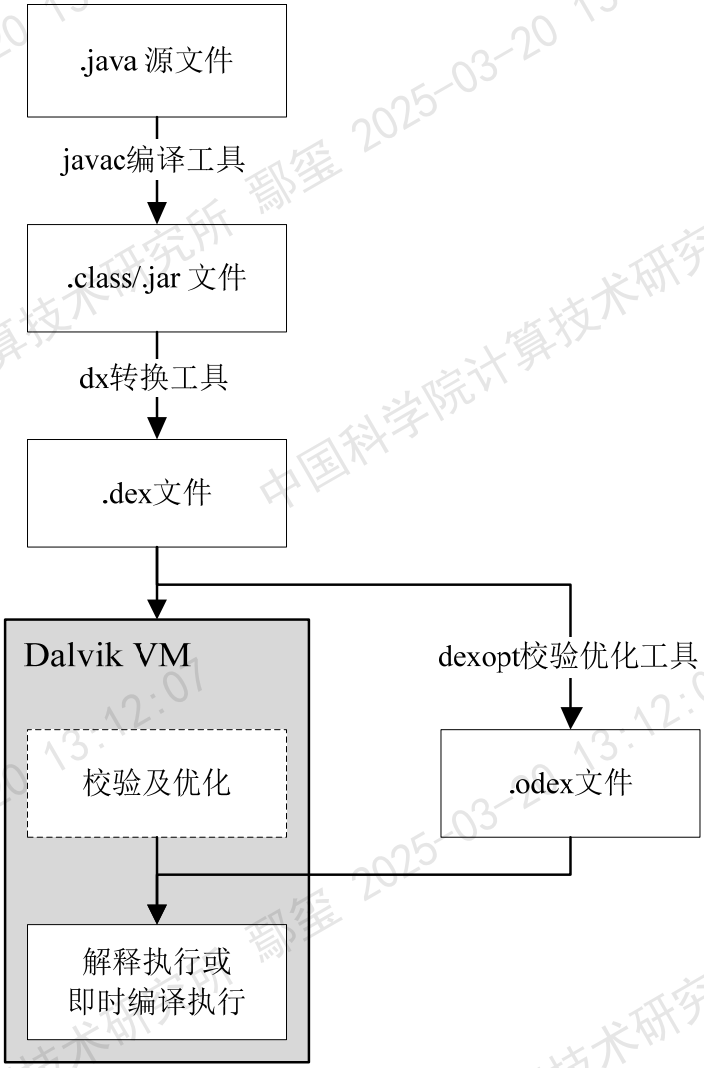

图 2-3 基于 Dalvik 虚拟机的 Java 程序运行流程

dx 工具转换文件格式分为以下两个步骤：一是将输入文件中的标准 Java 字节码指令翻译为 Dalvik 虚拟机特有的字节码指令集，二是按照 dex 格式规范对输入文件的各字段重新进行组织和布局；经转换得到的 dex 文件体积相比输入的 class 文件将显著缩小[19]。如图 2-4 是经转换后的 dex 文件与原始 jar 文件的布局对比。

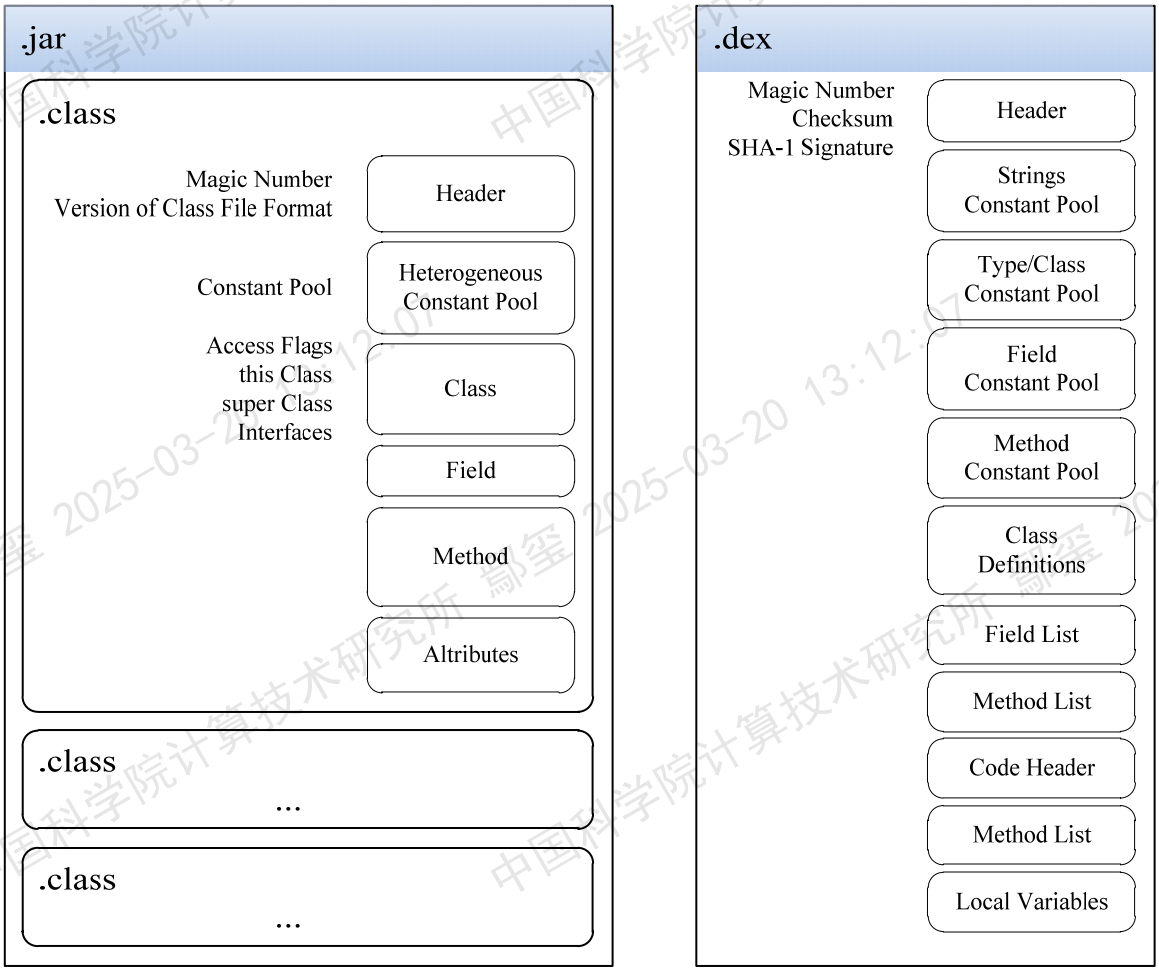

图 2-4 jar 文件与 dex 文件的结构对比

一般 dx 工具转换后的输出文件就可以直接载入 Dalvik 虚拟机运行了。在 Dalvik 虚拟机初次加载某个 dex 文件时，会首先进行文件校验，目的是预先检查输入程序中的非法指令序列；校验通过后虚拟机还会对输入程序进行虚方法索引替换、简单方法自动内联等进一步优化，最后将优化后的程序交由解释器执行。

由于虚拟机在程序加载时的校验和优化工作需要耗费一定的时间，影响程序的启动速度，所以 Dalvik 还提供了专门的 dexopt 二进制工具，开发者利用该工具可以预先完成文件的校验和优化，工具会在校验优化后输出的 odex 文件中添加相关标记，Dalvik 虚拟机在发现此标记后会直接跳过加载时的校验优化流程，从而加快了程序启动速度。

2.4.2 Dalvik 虚拟机的架构特点

Dalvik 虚拟机之所以能够实现比传统 JVM 更低的系统资源占用率，其最核心的因素在于 Dalvik 独有的指令集架构设计。

传统的 JVM 虚拟机均采用基于栈的指令集架构（stack-based JVM）设计，即

Java 字节码指令中不含有源操作数地址和目的操作数地址，因此也可以将该种指令称之为零地址指令。零地址指令的操作数都隐含地保存在求值栈（evaluation stack）中，在解释执行时需要首先从求值栈中弹出所需的源操作数，再交由 CPU 的运算单元进行相应运算，最后再把目的操作数压入求值栈。

采用基于栈的指令集架构设的最大优点在于指令形式简单，指令编码一般只需要 1 个字节来表示，因此程序中指令序列的排布更为紧凑；且在各种不同的处理器硬件架构上，都可以较容易地实现对零地址指令功能的模拟，因此该架构可以简化虚拟机内部实现，并具有较好的可移植性[20]。但其缺点就在于零地址指令的执行需要更多内存访问操作，且大多数现代处理器都采用基于寄存器的硬件架构设计，零地址指令也不易针对目标平台进行直接的指令映射和优化，因此指令的执行效率难以提高。

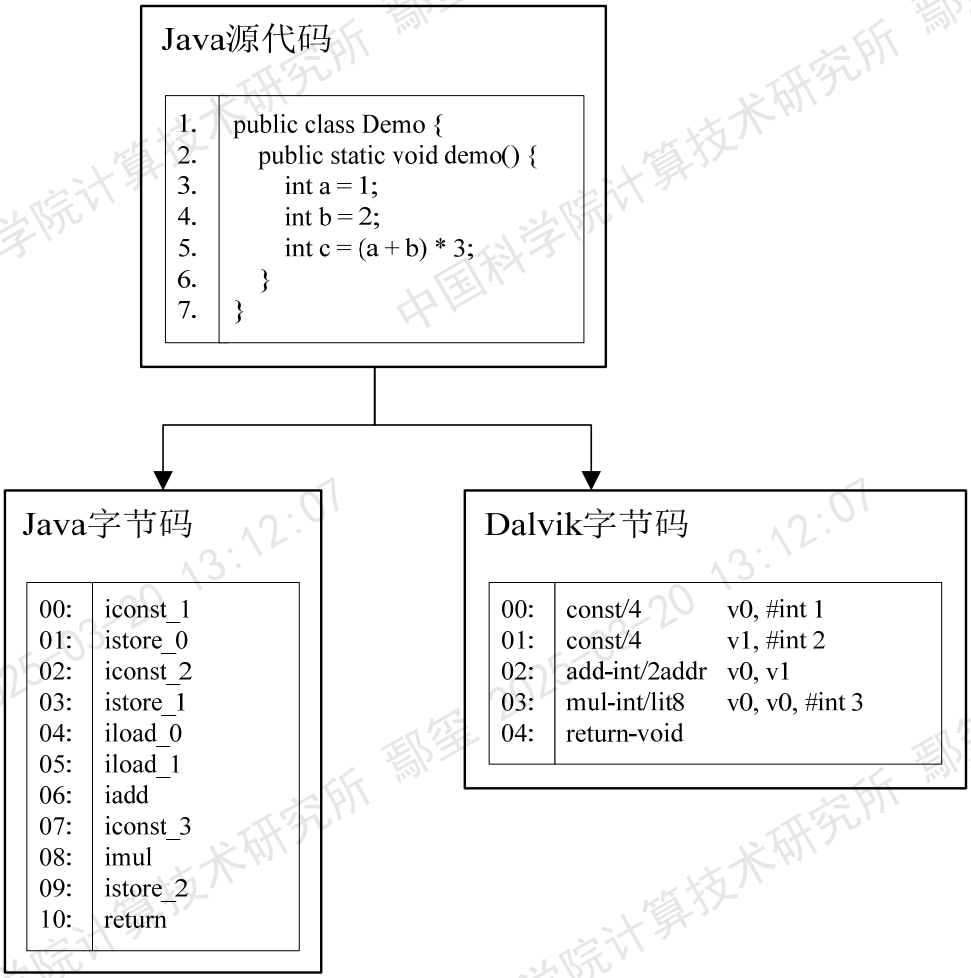

图 2-5 Java 字节码与 Dalvik 字节码格式对比

如图 2-5 所示，是同一段 Java 源代码在编译后生成的 Dalvik 字节码与 Java 字节码的对比[21]。Dalvik 虚拟机采用的是基于寄存器的指令集架构（register-based instructions）设计[22]，其指令编码以 2 个字节为单位，即至少需要 16 位来表示一条指

令。Dalvik 字节码指令支持显式的使用虚拟寄存器，一般在方法被调用时，被调用者可以获得 v0 ~ v15 共 16 个虚拟寄存器的独占使用权，每个寄存器的宽度为 32 位，该方法内的指令序列通过操作这组虚拟寄存器实现相应功能[23]。如果本地平台上的处理器实现了多个通用寄存器，则虚拟机在方法调用时，可以很容易地将虚拟寄存器直接映射为通用寄存器，在此基础上将字节码解释为等价的本地机器码。

对比之前的分析可以看出，基于寄存器架构的优点在于没有频繁访问求值栈的过程；而且由于单条指令的功能集成度比零地址指令更高，实现等价的程序功能所需的指令数会明显减少，加之基于寄存器的字节码指令对本地指令的映射效率更高，使程序在虚拟机上执行时的效率更高。其缺点则在于虚拟机实现与处理器硬件的耦合度更高，对于通用寄存器数量较少的处理器，模拟多个虚拟寄存器将会使虚拟机的内部实现更为复杂。

以上分析探究了 Dalvik 虚拟机的基本特点，为 Android 系统移植中与 Dalvik 相关的移植工作提供原理支持。从 Dalvik 虚拟机的内部架构可以看出，在移植时需要特别关注其中的字节码解释器模块与 JIT 编译器模块，这两个模块将实现字节码到目标平台的本地机器码的转换，所以其中包含大量体系结构相关的实现。在后续介绍 Dalvik 虚拟机 JIT 编译器移植的相关章节中，还将结合龙芯平台的具体热点对该部分内容进行阐述。

2.5 Android Java 核心库与应用程序框架

Android Java 核心库与 Dalvik 虚拟机一起构成了 Android 平台特有的 Java 运行时环境，其中 Android Java 核心库实现了传统 Java 运行时环境核心库的大部分功能，这使得其他系统平台的 Java 应用开发方法和技巧能够平滑地迁移至 Android 平台。

Android 的一大特色是在 Java 运行时环境之上提供了一套强大的应用开发框架（Application Framework），该框架在内部实现了活动管理器、通知管理器、UI 构件等一系列丰富的系统服务和常用组件，并以 Java API 形式将这些组件提供给应用开发者使用，与此同时还要求开发者遵循一定的应用开发模型[24]。通过这种方式，既使常用组件实现了高效重用，简化了开发者的工作；同时又规范了应用程序的行为，保障了系统的安全性和可恢复性。Android 系统在版本升级时会对 API 进行升级，以向应用开发者提供系统实现的最新功能，如 ICS 的 API 号为 14；一般而言新版本的 API 会尽可能保持向旧有版本的兼容性。

Android 平台上的应用均使用 Java 作为开发语言，开发者需要使用 Google 提供的可在 Windows/Mac OS/Linux 平台下使用的跨平台开发工具包（SDK，Software Development Kit）来进行开发，SDK 可以实现对框架层 API 功能的模拟，并提供了完

善的应用调试、发布工具。

由于 Java 核心库、应用程序框架以及具体的 Android 应用都使用 Java 语言开发，所以一般也将这三层统称为 Java 层。上文已经介绍，虽然在 Java 层中的程序会通过虚拟机的 JNI 机制调用在 C/C++层中实现的本地方法，其中某些本地方法具有平台相关性；但由于 Java 语言本身具有完全的跨平台性，因此在移植时只需要重新编译部署即可，无需对使用 Java 实现的源码进行修改。

2.6 本章小结

本章对 Android 系统的层次化设计思想以及整体架构进行了深入剖析，其中重点阐述了 Android Linux 内核定制驱动与 Android 系统本地共享函数库的功能作用，同时详细分析了 Davlik 虚拟机的执行流程以及内部架构特点，最后对 Android 应用框架及应用开发特点进行了介绍。

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

第三章　基于龙芯 3A 平台的 Android 系统移植

通过对大量系统移植类问题进行总结和分析，本课题将此类问题的一般解决方法归纳为图 3-1 所示的六个环节。

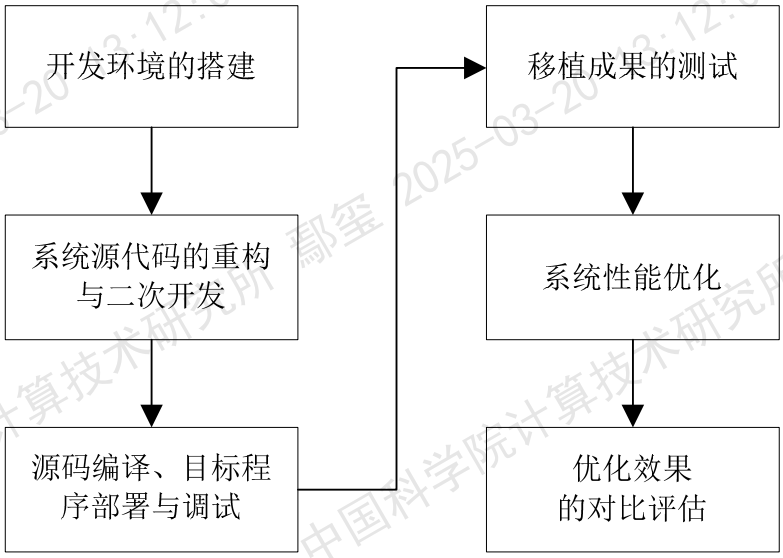

图 3-1 系统移植类问题的一般解决方法

本课题的相关工作即围绕以上六个环节展开。由于篇幅所限，本文只阐述其中的源码重构与二次开发、移植成果测试、系统性能优化以及优化效果评估四个部分。在本章中将着重介绍针对龙芯 3A 平台的 Android 4.0.4 系统源码重构与二次开发工作。

3.1　目标平台硬件配置概述

本课题旨在研究向龙芯 3A 平台移植 Android 系统时会遇到的平台相关问题及其解决方法，并不以产品原型开发或设备驱动调试为主要目的，因此不需要目标平台能支持 Android 系统的全部特性。鉴于此本课题选择了龙芯 3A780E 参考开发系统作为移植目标平台。该套硬件系统涵盖运行 Android 系统所需的最小系统并满足最低配置要求，该系统的具体构成如下：

（1）龙芯 3A 处理器，主频范围 600MHz~1GHz，集成 DDR3-800 内存控制器

（2）AMD 780E 高性能嵌入式平台北桥芯片组，集成 Radeon HD 3200 图形处理单元与 VGA 接口控制器

（3）SB710 高性能嵌入式平台南桥芯片组，集成 SATA 3Gb/s 接口控制器、USB2.0 接口控制器

（4）RTL8111DL 10Mbps/100Mbps/1000Mbps 网络控制器

（5）板载接口：4 个 DDR3-800 DIMM 内存插槽、1 个 VGA 接口、6 个 SATA 3Gb/s 接口、4 个 USB2.0 接口、1 个 RJ45 接口

在本课题的实验环境中，将处理器工作频率设定为 900MHz，并为开发系统配备了 2GB DDR3 内存，500GB SATA 3Gb/s 硬盘，100Mbps 网络连接；但尚未配备触摸屏、Flash 存储设备以及无线网络设备。

3.2 Android Linux 内核移植

3.2.1 Android 定制驱动的移植

在 2.2 节中已经介绍，Android 操作系统采用 Linux 作为系统内核，并且在 Linux 内核上添加了多个定制的驱动模块，因此本课题首先从 Android linux 内核向龙芯 3A 平台的移植展开相关工作。

向新型硬件平台上移植 Linux 内核的传统技术路线，是以某个稳定版本的 Linux 内核源代码为基础，针对目标平台重新编写其中体系结构相关部分的代码。这部分代码主要位于 Linux 内核源码树中的“arch/”目录与“include/asm-ARCH_NAME/”目录下，代码编写完成后还需要修改编译相关的 Kcongfig 文件和 Makefile 文件，再经编译链接之后即可生成在目标平台上运行的 Linux 内核[25]。

由于 Android 对 Linux 的定制部分以独立驱动模块的形式实现，大部分新增代码具有体系结构无关性，所以理论上依照上述技术路线完全可以将 Android Linux 内核移植至新的硬件平台。如具体在本课题中，可以首先通过对比龙芯 3A 版 Linux 内核相对于标准 Linux 内核的源码修改，检出龙芯 3A 平台的体系结构相关代码，再将其整合到选定的 Android Linux 内核源码中。

除上述技术路线外，对于已经可以成功运行某版本 Linux 内核的目标平台，考虑到 Android 定制部分模块化的特点，还可以选择类似 Linux 设备驱动移植的技术路线，即将 Android 系统定制的驱动模块整合至目标平台现有 Linux 内核。这部分定制驱动的源代码主要位于 Android Linux 内核源码树中的“drivers/staging/android”目录下，此外在“drivers/”以及“fs/yaffs/”目录下还有若干 Android 平台特有的硬件设备或文件系统驱动，可以根据目标平台的具体配置进行相应裁剪。根据本课题的需要，依照该路线的主要工作就是将“drivers/staging/android”目录下的 Android 核心驱动及其依赖的相关源文件整合至龙芯 3A 版 Linux 源码中。

从软件测试理论的角度对以上两条技术路线进行对比分析，由于针对增量开发进行的调试和测试，其核心工作将集中在增量代码部分[26]；因此可以预见，第一条技术路线的调试工作主要集中在“arch/mips/loongson”目录下的体系结构相关实现，而第

二条技术路线的调试工作集中在“drivers/staging/android/”目录下的新增驱动模块。

体系结构相关实现位于 Linux 内核架构的最底层，是其他各模块乃至软件调试环境正常运行的共同基础，所以这部分代码的可调试性较差；而具体的驱动模块与内核核心模块的耦合度较低，具有较好的可调试性，因此后者在可调试性上具有显著优势。而前者的优点在于体系结构相关代码只依赖于硬件实现，因此对上层模块的开发演变不敏感，版本稳定性较好；而驱动模块则依赖底层提供的各类接口，所以如果想在基线 Linux 版本更新时保持与基线的同步性，则需要反复进行迭代开发。

根据上述分析，结合本课题进行系统原型研究的移植目标，最终采用第二条技术路线进行 Android Linux 内核的移植，具体步骤如下：

（1）对比 Android Linux 分支与 Linux 基准分支的源码差异，根据龙芯 3A780E 平台的实际配置，从中剔除掉目标平台尚不支持的 YAFFS 文件系统驱动、无线网络设备驱动等驱动模块，将“drivers/staging/android/”、“drivers/base/”、“drivers/switch/”、“include/linux/”、“kernel/power/”等目录下的 Android 核心驱动整合至龙芯 3A Linux 源码的对应目录下。

（2）修改龙芯 3A Linux 源码中与以上目录及代码相关的 Kconfig 文件和 Makefile 文件，将新增加的源码文件添加到内核编译系统的依赖目标中。

（3）在 ashmem/binder/alarm-dev 三个 misc 设备驱动程序中实现 compat_ioctl 方法，并将函数指针添加至设备结构的操作接口表中，从而实现 64 位内核驱动对 32 位用户空间程序的兼容，后文将详述其原理与实现方法。

（4）编辑 Linux 内核编译配置文件即.config 文件，开启 CONFIG_ANDROID、CONFIG_COMPAT 等与 Android 驱动编译相关的配置选项。

（5）使用针对龙芯平台的编译工具链进行编译，生成 vmlinux 内核镜像文件。

3.2.2 Android 定制驱动的 64 位兼容模式开发

由于 Android 系统底层实现以及其编译工具链的限制，目前在所有平台上，Android 文件系统均只支持 32 位编译，即用户空间程序只能采用 32 位模式；而由于龙芯 3A 兼容 MIPS64 指令集，所以目前龙芯 3A 平台上全部使用 64 位 Linux 内核，因此 Android 定制的驱动模块在整合至龙芯 3A 内核后，亦会被编译为 64 位位宽。因此本课题需要特别考虑 64 位内核中的 Android 定制驱动对于 32 位用户空间程序的兼容性问题，这是与其他平台所不同的。

参考 2.2 节中对 Android 定制驱动模块的分析，可以发现上述兼容性问题集中在 ashmem/binder/alarm-dev 这四个 misc 杂项设备驱动中的 ioctl 接口上。

具体而言，下面以 ashmem 为例，详细解释驱动程序的加载以及 ioctl 接口的调用流程。

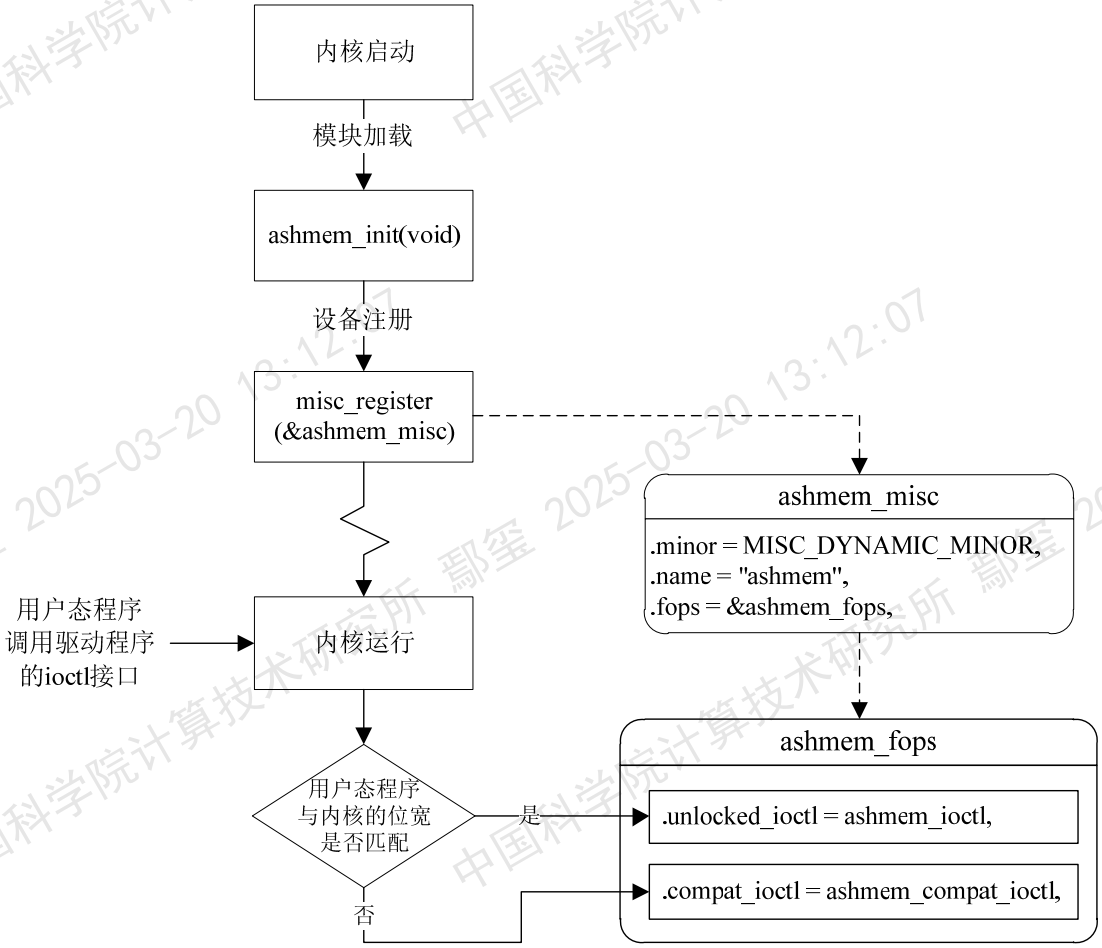

图 3-2 Linux 中 ioctl 调用的执行流程

如图 3-2 所示，驱动程序的加载和调用流程如下：

（1）内核启动后进行各项系统初始化工作，直至进入驱动程序加载环节

（2）在驱动程序加载环节中，ashmem 驱动程序被内核加载，此时程序执行流首先进入__init ashmem_init(void)驱动初始化函数

（3）在驱动初始化函数中，通过调用 misc_register(&ashmem_misc)设备注册函数，驱动程序向内核注册 ashmem_misc 这一设备核心数据结构，该数据结构的定义如下。

代码 3-1 ashmem_misc 数据结构定义

```
struct miscdevice ashmem_misc = {
        .minor = MISC_DYNAMIC_MINOR,
        .name = "ashmem",
        .fops = &ashmem_fops,
};
```

其中 ashmem_misc.fops 域指向一个函数表，即 file_operations 类型的数据结构 ashmem_fops；此表中记录了 ashmem 驱动所有操作接口的函数指针，ashmem_fops 中各域的定义如下。

代码 3-2 ashmem_fops 数据结构定义

```
1.  struct file_operations ashmem_fops = {
2.          .owner = THIS_MODULE,
3.          .open = ashmem_open,
4.          .release = ashmem_release,
5.          .read = ashmem_read,
6.          .llseek = ashmem_llseek,
7.          .mmap = ashmem_mmap,
8.          .unlocked_ioctl = ashmem_ioctl,
9.  };
```

（4）用户空间程序需要通过 ioctl 系统调用来调用 ashmem 驱动提供的 ioctl 接口；首先程序执行流将陷入内核中的 ioctl 系统调用处理函数，该函数通过查询已注册设备的链表，获得 ashmem_misc 这一数据结构；最终根据该数据结构中的方法表入口也即 fops 指针，实现对驱动程序中 ashmem_ioctl 接口的实际调用。

根据 linux 内核现有的用户空间位宽兼容机制，当用户空间程序的位宽内核位宽一致时，ioctl 调用最终将调用驱动程序中的 unlocked_ioctl 接口；而当用户空间程序的位宽内核位宽不一致、也即 32 位用户空间程序调用 64 位内核的 ioctl 系统调用时，最终将调用驱动程序中的 compat_ioctl 接口[27]。

由此可见，为了保证 64 位的 Android 驱动程序对于 32 位用户空间程序的兼容性，需要在 ashmem/binder/alarm-dev 三个驱动程序中添加并实现 compat_ioctl 接口。

compat_ioctl 接口是驱动程序中已有的 unlocked_ioctl 接口的 64 位兼容实现，即 compat_ioctl 接口的逻辑功能与 unlocked_ioctl 接口完全等价，而 compat_ioctl 中不能使用任何对编译位宽敏感的数据类型来声明变量。

在 Linux 内核所使用的编译工具链中，指针、long、unsinged long 三种数据类型在 32 位编译模式与 64 位编译模式下所占用的内存大小会存在差别，如表 3-1 所示。

表 3-1 数据类型在 32 位与 64 位编译下的内存占用对比

类型	字节数	
	32-bit 编译	64-bit 编译
char	1	1

*（指针类型）	4	8
short int	2	2
int	4	4
unsigned int	4	4
float	4	4
double	8	8
long	4	8
long long	8	8
unsigned long	4	8

因此在实现新的 compat_ioctl 接口时，需要以 unlocked_ioctl 接口的实现为蓝本，将 unlocked_ioctl 接口中的指针类型，long，unsinged long 三种类型的变量，全部改声明为对编译位宽不敏感的数据类型。如图 3-3 是以 alarm-dev 驱动为例，对比 compat_ioctl 接口与 unlocked_ioctl 接口实现的区别。

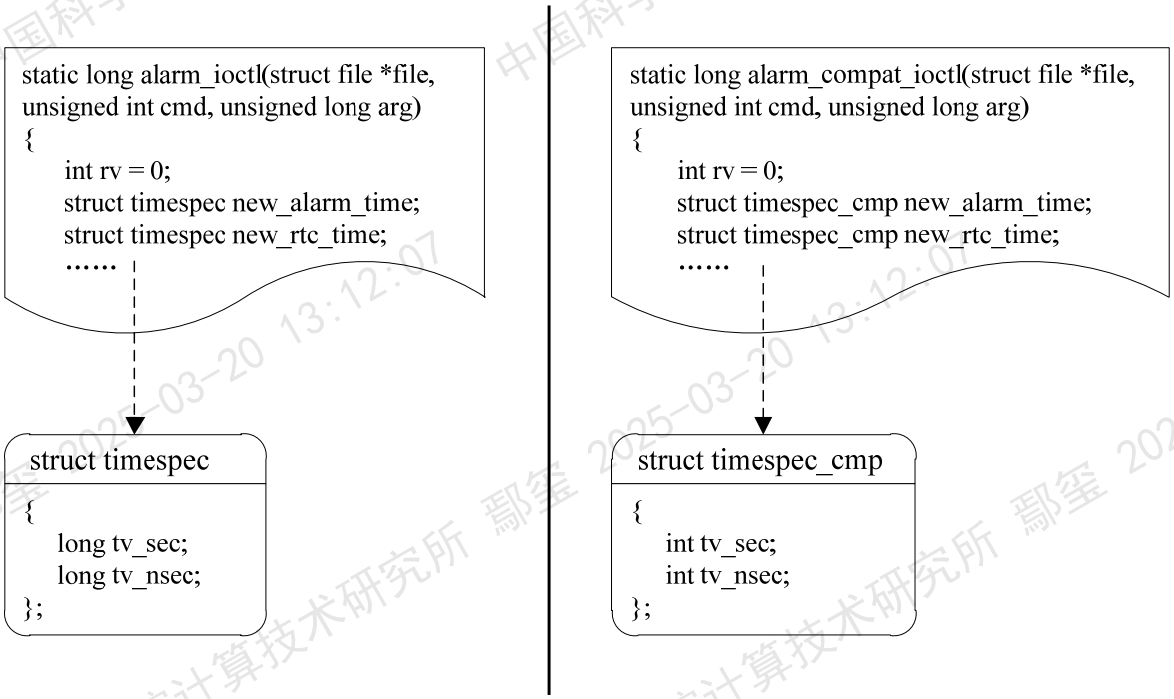

图 3-3 compat_ioctl 接口与 ioctl 接口的实现对比

在完成 compat_ioctl 接口的实现后，还需要将接口的函数指针保存到设备核心数据结构的接口表中，如图 3-4 所示。

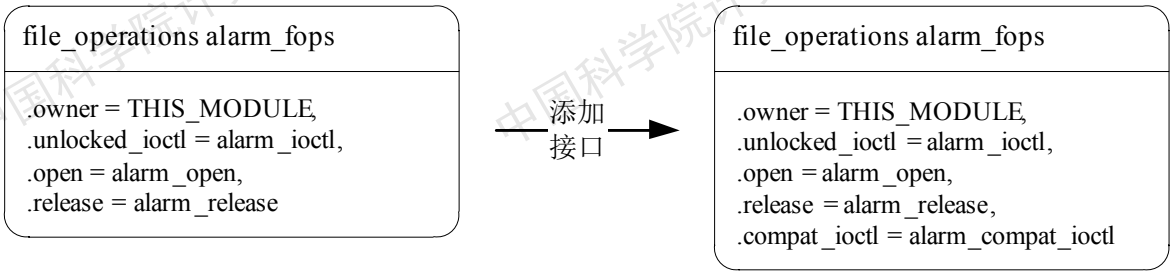

图 3-4 compat_ioctl 接口在驱动接口表中的添加

经以上重构后即可保证，采用 64 位编译的 compat_ioctl 接口，与采用 32 位编译的 unlocked_ioctl 接口在逻辑功能上完全等价；因而实现了 64 位内核中的 Android 驱动对 32 位用户空间程序的兼容。

3.3 针对龙芯 3A 的 Android 编译系统构建

在本节首先介绍针对龙芯平台搭建 Android 系统编译环境的相关工作，为下节详述 Android 源码的修改重构工作提供概念支撑。

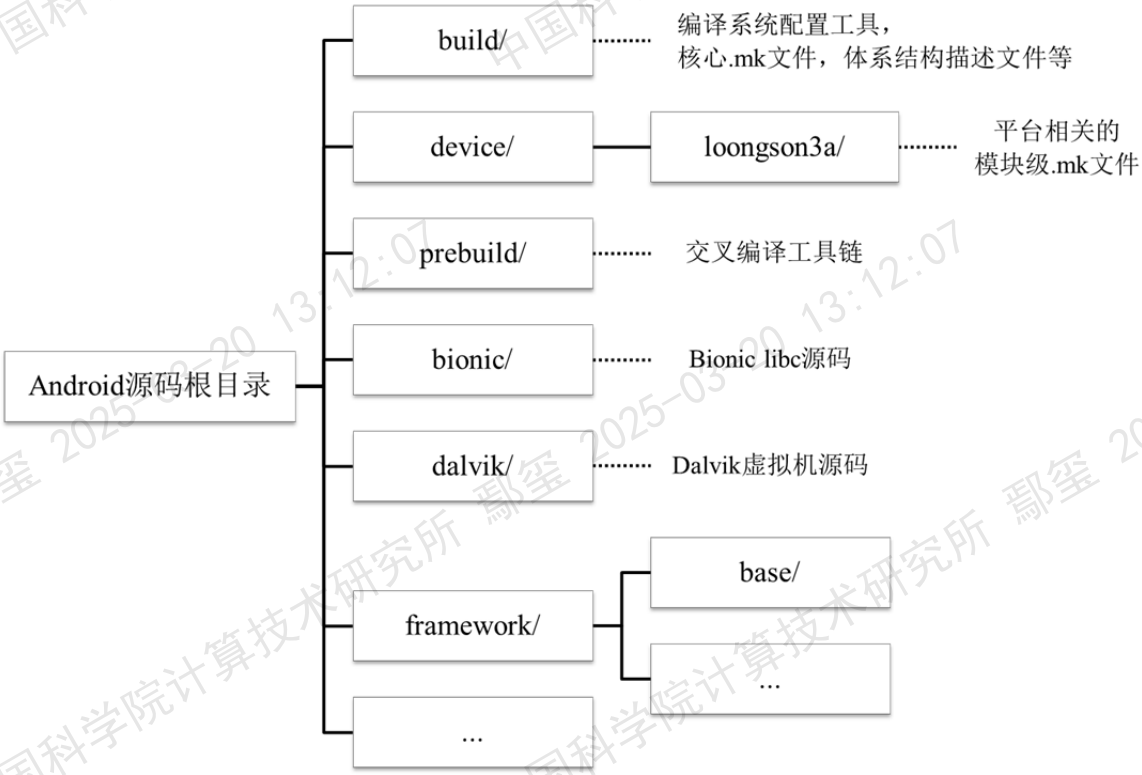

图 3-5 Android 源代码组织结构

Android 开源项目与大多数大型软件项目一样，采用多级目录树的形式组织项目的源码文件。完整的 Android 开源项目由 200 多个开源子项目组成，每个子项目的源

码都被布置在源码目录树的某一级子目录下，如图 3-5 所示，Bionic libc 项目位于目录树的一级子目录“binoic/”下，framework-base 项目位于二级子目录“framework/base/”下，以此类推。其中用于搭建编译环境的项目是 build 和 device、prebuild，实现 Android 编译框架的编译配置文件位于 build 目录下，与目标平台相关的编译配置文件位于 device 的二级目录下，交叉编译工具链在 prebuild 目录下。

Android 与 Linux 内核一样使用 GNU Make 工具作为其编译系统的核心。AOSP 官方发布的 Android 编译指南中只简单描述了以下两个编译步骤：第一步使用 envsetup.sh 编译系统配置工具以及 lunch 命令进行编译环境的自动化配置；第二步在 Android 源码根目录下执行 make 命令进行自动化编译。事实上在执行上述两步操作之前，Android 系统移植者必须预先向编译系统提供目标平台的体系结构描述文件、外设模块列表以及交叉编译工具链等多项配置文件及工具。具体而言，根据目标平台的配置情况，本课题分以下几个步骤进行了编译环境的搭建工作。

（1）建立 device/loongson/Loongson3A_Platform/ 目录，在该目录下创建 vendersetup.sh 脚本文件，该文件部分关键内容摘录如下。

代码 3-3 vendersetup.sh 脚本文件内容摘录

```
1.  export TARGET_CPU_ABI=mips
2.  export TARGET_ARCH=mips
3.  export TARGET_ARCH_VARIANT=loongson3a
4.  add_lunch_combo Loongson3A_Platform-eng
```

在后续的编译环境设置过程中，第（5）步的 lunch 命令会根据该脚本中的 TARGET_CPU_ABI、TARGET_ARCH 等变量，加载目标平台体系结构的详细描述文件。

（2）在“build/core/combo/arch/mips/”目录下创建龙芯 3A 处理器的体系结构描述文件 loongson3a.mk，文件部分关键内容摘录如下。

代码 3-4 loongson3a.mk 文件内容摘录

```
1.  ARCH_MIPS_HAS_FPU :=true
2.  TARGET_CPU_SMP := true
3.  arch_variant_cflags := -EL -march=mips32 -mhard-float -DPAGE_SIZE_16KB
```

在上一步创建的 verndersetup.sh 脚本中，已通过“TARGET_ARCH_VARIANT = loongson3a”变量指明 loongson3 处理器为 MIPS 体系结构的一种实现，因此在第（5）步中执行 lunch 命令时，会自动进入“build//core/combo/arch/mips/”目录下加载 loongson3a.mk 文件，并根据文件中描述的龙芯 3A 处理器含有硬件浮点部件、SMP

多核、小尾端以及使用 16KB 内存页等特性，对交叉编译工具链进行配置。其中需要特别指出的是，在 3.2.2 中已提及，Android 系统尚不支持 64 位位宽的编译，因此在该文件中将龙芯 3A 的 MIPS 细分指令集类型标记为 mips32，交叉编译工具会据此编译生成使用 mips32 指令集 32 位的 Android 文件系统。

（3）在“device/loongson/Loongson3A_Platform/”目录下创建 BoardConfig.mk、device.mk 等编译配置文件，这些文件主要描述了目标平台的模块级信息，如图形处理单元、WIFI 模块等设备的型号。当编译工具进行这些设备相关模块的编译时，会根据以上配置文件中描述的参数，寻找对应的源文件并对源码中的预编译信息进行处理。

（4）将“prebuilt/linux-x86/toolchain/”目录下的 MIPS 平台通用交叉编译工具链，替换为龙芯定制的交叉编译工具链。Android 系统与其他软件项目的一个重要区别是，Android 编译系统不使用开发平台上现有的编译工具链，而是会使用源码中“prebuilt/”目录下自带的交叉编译工具链。MIPS 版 Android 源码中自带的编译工具链为 MIPS 平台的通用版本，该工具链不能完全兼容龙芯 3A 平台，因此必须将其替换为龙芯定制的交叉编译工具链，才能保证编译生成的目标文件在龙芯 3A 平台上正确运行。

（5）加载 build/envsetup.sh 脚本文件，并执行“lunch Loongson3A_Platform-eng”命令，在该命令执行成功之后，针对龙芯 3A 平台的编译环境搭建工作业已完成。

下面将重点阐述本课题针对龙芯 3A 平台进行的 Android 源码修改与重构工作，在源码修改完成后执行 make 命令，即可编译生成可在龙芯 3A 平台上运行的 Android 文件系统。

3.4 Bionic 与 Dalvik 的移植

Bionic libc 库和 Dalvik 虚拟机作为 Android 系统的两个底层核心模块，其实现中有多处具有目标平台相关性，因此需要对二者的源码进行相应修改。其修改内容主要包括以下三方面内容。

（1）Bionic 内存页大小的修改

内存管理是 Bionic 库实现的一个重要功能，用户空间程序可以使用经过 Bionic 库封装的接口完成内存的申请释放操作。对基于分页机制来进行内存管理的处理器和操作系统来说，内存页大小（page size）这一参数的正确配置是实现正确的内存管理工作的前提。在原版的 Bionic 库源码中，采用预编译宏的方式将内存页大小设定为 4KB，而龙芯 3A 处理器只支持使用 16KB 的内存页，因而需要对其进行修改[12]。

需要说明的是，Bionic 源码中实际是通过 PAGE_SHIFT 宏来计算内存页大小，即 PAGE_SIZE = 1 << PAGE_SHIFT，因此 PAGE_SHIFT = 14 时，PAGE_SIZE = 16KB。

（2）Bionic 对 SMP 架构内存屏障操作的支持

在 SMP 架构的多核处理器上进行程序开发时，经常需要使用本地共享函数库所提供的原子化内存屏障（memory barrier）操作，来保证多核心之间访存的顺序性。龙芯 3A 是基于 MIPS 体系结构的 4 核心 SMP 架构处理器，但原版的 Bionic 库中只实现了针对 ARM 体系结构和 X86 体系结构的内存屏障操作，并没有 MIPS 体系结构的相应实现，因此本课题专门添加了这一实现，新添加内容如下。

代码 3-5 Bionic 中新增的内存屏障操作定义

```
1.    #elif defined(__mips__).
2.    # define _ANDROID_MEMBAR_FULL_SMP() \
3.    do { __asm__ __volatile__ ("sync" ::: "memory"); } while (0)
4.    #else
5.    …
```

该宏定义中使用了内嵌的 MIPS 汇编指令 sync，即用于保证多核心间访存操作的强有序性[28]。

（3）Dalvik 内存页大小的修改

与 Bionic 库相同，Dalvik 虚拟机的源码中所涉及的内存页大小参数也需要被修改为 16KB，以保证与龙芯 3A 的兼容，修改方法与 bionic 相似故不再冗述。

3.5 Android 图形子系统的重构

目前在手持移动设备上，单纯使用通用处理器负责全部 2D/3D 图形的计算、渲染、绘制工作，已经无法满足交互式系统的性能要求。因此大量移动设备都配备了硬件图形处理单元，专门用来处理操作系统中图形子系统的计算任务。龙芯 3A 平台上亦配备了 AMD Radeon HD 3200 图形处理单元，该单元集成在北桥芯片组中，其图形处理核心的型号为 R600，支持 OpenGL2.0 图形程序接口。

在现有的 Android 图形子系统框架中，由 Surfaceflinger 服务向应用程序提供绘图流的合成、图形渲染、缓冲管理等服务，而这些服务的实现则需要调用 Android 本地共享库中提供的 OpenGL 接口和 EGL 接口[29]。

如图 3-6 所示的 Android 图形子系统的 Server 端架构，在 Android 本地共享库中包含纯软件和硬件加速这两种 OpenGL 系列接口的实现。其中纯软件实现，即 libGLES_android.so 库的源码已包含在 Android 源码目录下，该库全部使用 C++语言

编写且不依赖任何图形处理硬件，因此具有面向各目标平台的通用性。

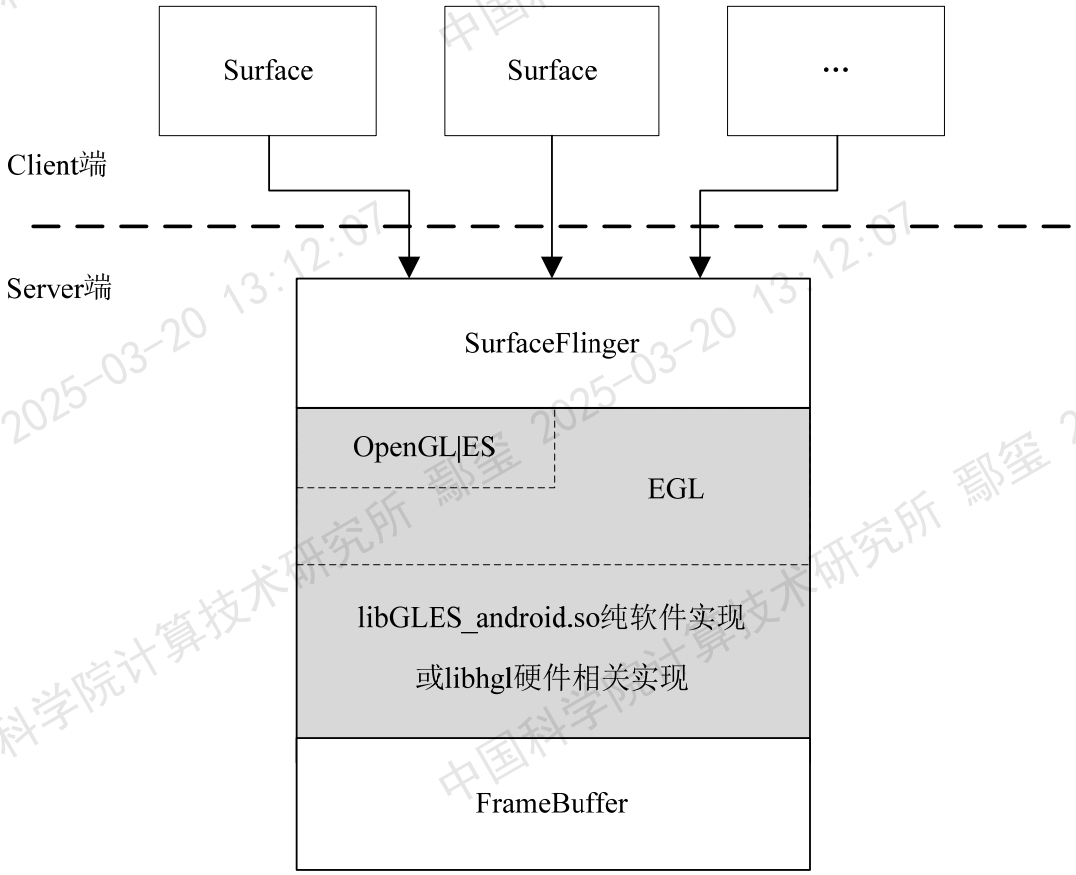

图 3-6 Android 图形子系统 Server 端架构

经本课题的实验测试发现，在将 Android 系统成功移植至龙芯 3A 平台后，使用 libGLES_android.so 库的图形子系统虽然可以正确工作，但其性能无法满足本课题在 1.2 节提出的对于系统流畅程度的目标要求，例如 Android 系统图形界面的响应延迟较高，图形显示会频繁出现用户可感知的卡顿。

理论上图形硬件加速实现的性能相比于纯软件实现会有较大幅度提升，但是在 Android 系统原有的图形子系统框架中，采用硬件加速的图形库中包含大量的图形硬件驱动程序，所以该库一般只由图形处理硬件的开发厂商以闭源的方式提供，独立的研究者和开发者也无法对其进行二次开发和移植。而目前 Android 平台上尚没有支持 R600 图形核心的闭源硬件加速库，因此，龙芯 3A 平台无法在原有的 Android 图形子系统框架下启用图形硬件加速。

基于以上原因，本课题选择 Mesa 3D 图形库作为龙芯 3A- Android 平台的 OpenGL 接口和 EGL 接口的实现方案。目前已移植到龙芯 3A 平台上的 Debian、Fedora 等操作系统均基于该库实现图形处理的硬件加速，且亦有开发者曾验证在 x86 Android 平台上使用 Mesa 3D 库实现图形硬件加速的可行性[30]。经过分析和实验，本

课题按照图 3-7 所示方案对龙芯 3A Android 平台的图形子系统 Server 端架构进行了部分重构。

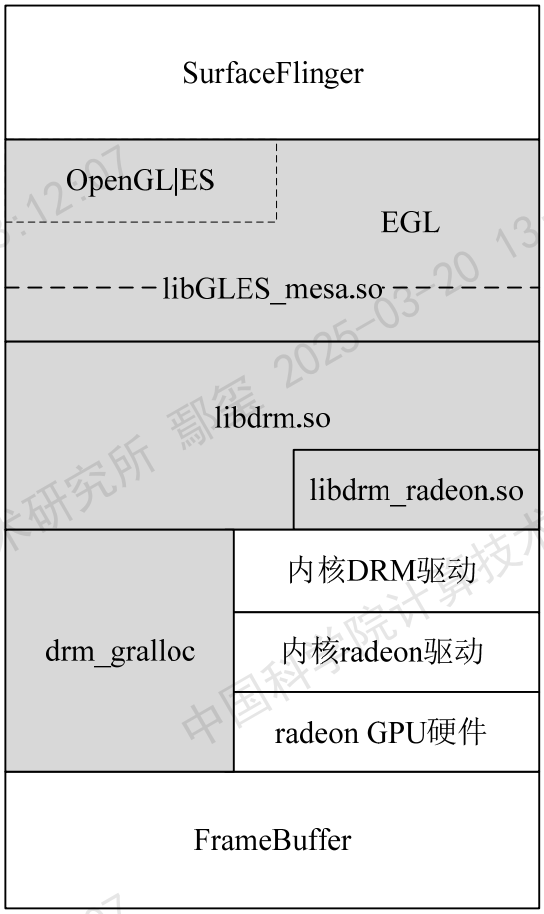

图 3-7 龙芯 3A-Android 系统采用的图形子系统 Server 端架构

重构后的图形子系统 Server 端由 libGLES_mesa.so、libdrm.so、libdrm_radeon.so 等几个库组成。其中 libGLES_mesa.so 库为 OpenGL 接口与 EGL 接口的适配层，为应用层提供 OpenGL 接口的调用；libdrm.so 库为直接渲染管理器（DRM，Direct Rendering Manager）的实现，其作用是为上层的接口适配层提供图形处理硬件的硬件抽象层（HAL，Hardware Abstraction Layer），因此该库依赖用户态的图形硬件驱动，在龙芯 3A 平台上即对应 R600 图形处理器单元的 libdrm_radeon.so 驱动；此外用户态的 drm_gralloc 模块实现了帧缓冲区的硬件抽象，即负责对上层提供帧缓冲区的操作接口以及显存管理等功能[31]。

以上模块构成了龙芯 3A Android 平台图形子系统的用户空间部分，除此以外还需要内核空间中的 DRM 驱动以及 radeon GPU 驱动与之配合，即可在龙芯 3A 平台上实现基于 radeon GPU 硬件加速的 Android 图形子系统。在第四章中将对图形子系统性能进行具体测试。

3.6 本章小结

本章首先将系统移植类问题的通用解决方法一般性地归纳为开发环境的搭建、系统源码的重构与二次开发、源码编译以及目标程序部署调试，移植成果测试、系统性能优化、优化效果的对比评估六个关键环节。然后按照上述方法论定义的次序，首先阐述了 Android 系统源码的重构与二次开发工作。其中 Android Linux 定制驱动的移植及其 ioctl 接口的二次开发，实现了龙芯平台上的 64 位 Linux 内核对 32 位 Android 文件系统的支持；通过在 build 等项目的新增相关配置文件实现了 Android 编译系统对龙芯 3A 平台的支持；通过 bionic 中的内存页大小以及 MIPS SMP 内存屏障原语的二次开发实现了 Bionic libc 对龙芯 3A 平台的支持；最后基于 Mesa 3D 图形库的 Android 图形子系统重构实现了龙芯 3A-Android 系统的图形硬件加速。

中国科学院计算技术研究所 鄢銮 2025-03-20 13:12:07

第四章　移植成果的测试与评估

上一章中已详细阐述了针对龙芯 3A 平台的 Android 源码移植以及编译环境搭建工作，在上述工作基础上编译生成的 Android 文件系统已经基本具备了在龙芯 3A 平台上的可用性。本章将继续按照 1.4 节中归纳的移植环节，重点阐述 Android 文件系统的部署以及在目标平台上的系统性测试工作。

4.1　Android 文件系统的部署

编译成功后，交叉编译生成的目标程序、配置脚本以及启动镜像文件会统一输出至开发平台的指定目录下。对于使用 Flash 存储器作为文件系统存储介质的目标平台，只需将启动镜像文件写入到 Flash 存储器中即可完成文件系统部署。而对于龙芯 3A 平台这种使用机械硬盘存储文件系统的目标平台，需要先将编译生成的目录、程序及配置脚本组织成 Android 根文件系统要求的目录结构，再把此根文件系统写入硬盘中。最终在硬盘根目录下部署完毕的 Android 文件系统结构如图 4-1 所示。

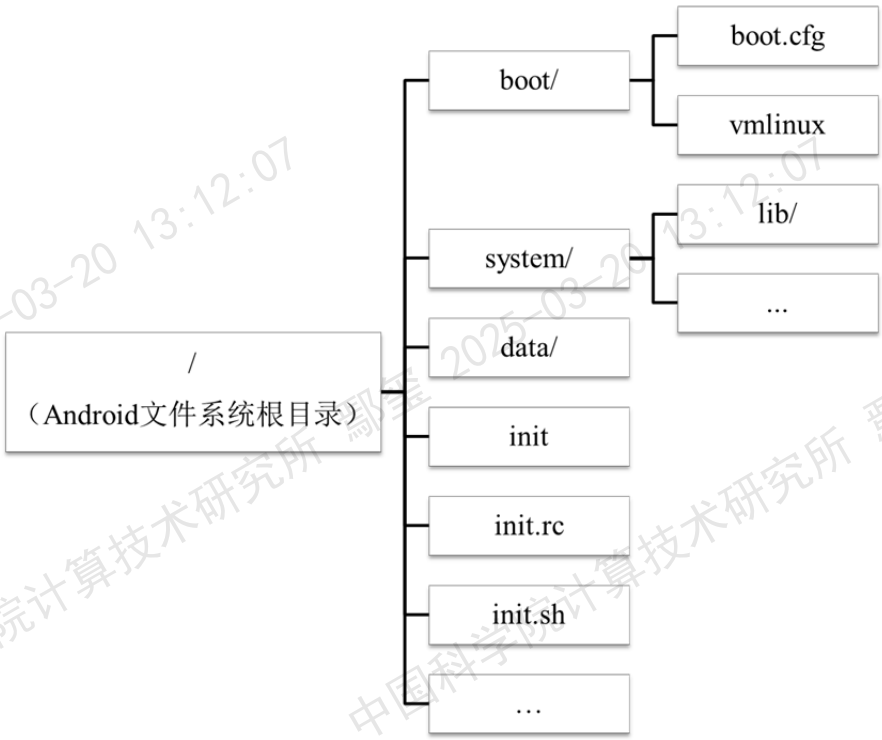

图 4-1 Android 文件系统组织结构

其中“system/”目录下主要存储系统程序、共享库与类库、以及原生应用程序；

“data/”目录下存储应用程序产生的相关数据；init 程序是 linux 内核加载根文件系统时装载的首个程序也即 init 进程；“boot/”目录非 Android 文件系统的组成部分，而是专门用于存储 Linux 内核及内核引导配置。

除此之外，在龙芯 3A 平台上实现 Android 操作系统的正常启动，还需要正确编写 boot.cfg 引导配置文件以及 int.rc、init.sh 初始化脚本文件。

boot.cfg 用于向 BIOS 提供引导内核时所需的引导参数，本课题采用如下配置进行 Android Linux 内核的引导。

代码 4-1 龙芯 3A-Android 内核引导配置

```
1.  kernel /dev/fs/ext2@wd0/boot/vmlinux-android
2.  args console=tty root=/dev/sda2 init=/init androidboot.console=tty
```

init.rc 脚本用于为 Android 操作系统启动后运行的首个程序，即 init 程序提供所需执行的初始化命令序列。init 程序会根据这些命令完成系统权限设置、系统属性值设置、系统服务启动等一系列操作。其中有多项命令的参数依赖于目标平台的配置，在初次启动时需要反复调试。下面仅以 Dalvik 虚拟机的配置命令简单举例。

代码 4-2 龙芯 3A-Android Dalvik 虚拟机配置

```
1.  setprop dalvik.vm.heapstartsize 128m
2.  setprop dalvik.vm.heapsize 256m
3.  setprop dalvik.vm.execution-mode int:fast
```

以上命令配置了 Dalvik 虚拟机的堆栈大小等运行参数，经本课题的反复实验发现，使用低于以上的配置会导致虚拟机运行状态不稳定，进而引发系统异常。

init.sh 脚本将在系统初始化结束阶段被执行，Android 系统移植者可以根据平台需要，在该脚本中自行添加初始化命令。例如龙芯 3A 平台采用有线以太网接口实现网络接入，因此需要在该脚本中添加 IP 地址配置等网络初始化命令。

经过上述的 Android 文件系统部署工作后，即可在龙芯 3A 平台上成功地启动并运行 Android 操作系统。

4.2 系统可用性的测试及评估

根据本文在第三章起始部分归纳得出的系统移植类问题的解决方法论，本节将首先介绍从普通的 Android 系统使用者的角度出发，对上述移植工作成果的基本可用性测试情况。如图 4-2 所示是测试过程中的部分场景记录。

在可用性测试中，除发现部分系统功能如电话拨打、无线连接等由于 3.1 节所述的目标平台外围硬件设备配置所限，尚无法使用以外，其他系统基本功能均可以正常

使用，且使用过程中的交互体验与 Android 操作系统的设计相一致，操作响应延迟满足交互式系统的流畅性要求。

图 4-2 龙芯 3A-Android 系统基本功能测试场景

此外本课题还使用 0xBenchmark 测试工具进一步测试了系统启用图形硬件加速后的 2D/3D 图形性能[32]，测试数据如表 4-1 所示，图 4-3 为测试时的运行场景记录。

表 4-1 龙芯 3A-Android 系统 2D/3D 图形性能测试结果

测试集	测试用例	测试帧率/fps
0xBenchmark 2D	Canvas	52.92
	Circle	31.29
	Rect	12.43
	Arc	13.92
	Image	7.72
	Text	39.06

0xBenchmark OpenGL 3D	Cube	25.87
	Blending	13.74
	Fog	14.02
	Teapot	16.20

*测试帧率取 3 次测试的算术平均值

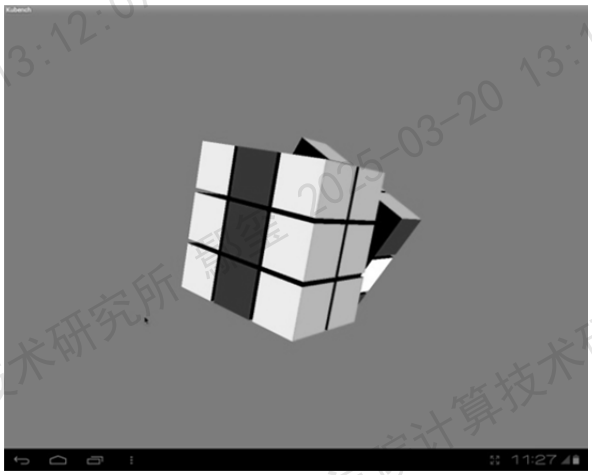

图 4-3 龙芯 3A-Android 系统 3D 图形性能测试场景

从表 4-1 的测试结果可见，各项 2D/3D 图形测试的平均帧率已经达到或接近一般认为的 15fps 的流畅显示基准，进一步验证了移植后系统已具备一定实用性。

4.3 移植系统的应用兼容性测试及分析

除系统的基本可用性测试以外，本课题还选取了多个较为常用的 Android 应用程序，对系统移植后对应用兼容性造成的影响进行了全面测试，表 4-2 为部分所测应用的兼容性情况，图 4-4 为测试浏览器应用时的运行场景记录。

表 4-2 龙芯 3A-Android 系统应用兼容性测试部分结果

	浏览器	通讯录	谷歌输入法	QQ HD	优酷视频	有道词典	百度地图	WPS	愤怒的小鸟
完全正常	√	√	√			√		√	
部分正常				√	√				
不可用							√		√

图 4-4 龙芯 3A-Android 系统浏览器测试场景

深入分析表 4-2 中部分应用无法兼容移植后系统的原因可以发现，某些应用存在部分功能异常是由于此类功能需要无线网络、GPS 等相关外围设备的支持，而龙芯 3A 平台尚未搭载以上外围设备。而部分应用完全无法使用的原因是其应用包中含有针对 ARM 平台的本地代码，也即应用开发时使用了基于 ARM 平台的 Java native 方法，因此该部分本地代码无法在 MIPS 上运行，从而造成应用整体无法兼容。

4.4 本章小结

本章首先简要介绍了 Android 文件系统目标程序在龙芯 3A 平台上的部署方法，然后对部署完毕的龙芯 3A-Android 系统进行了系统可用性、基本性能以及应用程序兼容性等方面的测试；通过实际使用以及运行相关性能测试集，验证了移植后系统已具备一定实用性；通过对多个应用程序的实际使用测试，发现移植后系统存在部分应用程序的兼容性问题，其主要原因是龙芯 3A 平台外围设备支持的局限以及应用包中本地代码具有平台相关性。

中国科学院计算技术研究所 鄢銮 2025-03-20 13:12:07

第五章　基于龙芯 3A 平台的 Dalvik 虚拟机优化

上述两章已详细阐述了本课题在 Android 系统移植及系统测试方面所完成的工作。在前述工作的基础上，本课题通过 Dalvik JIT 编译器的移植大幅度提高了龙芯 3A 平台上的 Dalvik 虚拟机性能；同时还提出并实现了适用于任意平台的基于复合评价模型的热度检测算法，采用该算法实现了对 Dalvik JIT 性能的进一步优化。

在本章中，将首先简单阐述 Android 应用性能模型的构建方法，分析说明 Dalvik 虚拟机优化对于 Android 应用性能提升的意义；然后对 Dalvik 虚拟机 JIT 即时编译技术的原理进行剖析；在基础上详细阐述两种优化方法——Dalvik JIT 编译器移植，以及基于复合评价模型的热度检测算法的设计思路与实现方法；最后通过性能对比测试对两种方法的优化效果作出评估。

5.1　Android 应用的性能模型构建及分析

在第二章中已介绍，Android 操作系统的应用框架和应用程序均采用 Java 语言开发，所有的 Java 程序均由 Dalvik 虚拟机来执行，对于部分使用了 Java native 方法的应用程序，会通过 Dalvik 虚拟机实现的 JNI 机制进一步调用由 C/C++语言编译生成的本地方法。 因此，在目标平台硬件环境固定的条件下，可以认为 Android 应用程序的性能由以下五个独立因素共同决定：

$$\text{Android Java 程序执行时间} = \text{Java 字节码数量} \times \text{Dalvik 执行平均耗时} + (\text{JNI 调用开销} + \text{本地方法耗时}) \times \text{JNI 调用次数} \qquad (5\text{-}1)$$

其中，Dalvik 执行字节码的平均耗时与 JNI 调用开销取决于 Dalvik 虚拟机的实现，是系统性能优化的重要切入点。在 1.3 节的本课题相关研究现状中已介绍，曾有研究者对 Dalvik JNI 调用桥函数的龙芯平台汇编优化问题进行相关研究；而 Dalvik 虚拟机的执行效率作为影响 Android 应用性能的核心因素，尚没有研究者针对龙芯平台上的 Dalvik 执行效率优化问题展开相关研究。

在诸多提升 Java 虚拟机执行效率的优化技术中，JIT（Just-In-Time）即时编译技术目前被公认为是一种效果较为显著的优化方法[33]；目前 Dalvik 虚拟机中业已实现了针对 ARMv7 指令集、x86 指令集以及标准 MIPS32 指令集的 JIT 即时编译模块，因此下文将从 Dalvik 虚拟机 JIT 即时编译模块的工作原理切入，进一步探究对该模块进行移植和优化的方法。

5.2 Dalvik JIT 即时编译技术剖析

在第二章中已经介绍，Java 程序基本的执行模式是由 Java 虚拟机对 class 文件中的字节码逐条进行解释执行。而 JIT 即时编译技术则在解释执行的基础上，引入了可以将字节码直接编译为目标平台本地代码的编译器。根据软件优化领域经典的 80-20 经验法则——20%的代码会占用 80%的程序执行时间，当 Java 虚拟机检测到程序中的某些字节码片段达到“热度”阈值后，会在解释执行的同时启动编译器对其进行编译；当再执行至这些字节码片段时，即可直接执行编译生成的本地代码，从而替代原先的解释执行方式。

编译执行相比于解释执行的主要优势在于：编译器工作时可以打破解释器只能逐条翻译字节码的限制，能够基于字节码片段进行局部优化，因此编译生成的本地代码在处理器寄存器的分配和使用上具有更高的效率，可以减少临时变量的使用也即减少了内存访问操作；某些复杂度较高的 JIT 编译器还可采用方法内联、拷贝去除（Copy Elision）、常量叠算（Constant folding）等优化技术进一步提高生成代码质量。

由于执行本地代码带来的时间节省要远大于编译带来的额外时间开销，所以启用 JIT 即时编译后程序会获得高达数倍的加速比；而使用 JIT 技术的代价则在于编译器运行及本地代码存储需要占用额外的内存和外存空间。Google 曾在 Google I/O 2010 会议上发布报告，称当时在 ARM Android 2. 1 上启用 Dalvik JIT 编译器后，测试程序可以获得平均 4 倍以上的加速，而额外的内存开销在 100kB 以下[34]。

Dalvik JIT 编译器目前实现了以下两种编译方式：

（1）基于 method 的方式：以程序中的方法作为编译单位进行编译。当虚拟机检测到某个方法或函数被调用的“热度”达到阈值时，启动编译器将该方法整体编译为本地代码，同时还会根据情况使用方法内联等全局优化方法[35]。

（2）基于 trace 的方式：以程序中的具体路径作为编译单位进行编译。在一个热点方法或函数中，由于条件分支的存在，可能同时存在执行的热路径与冷路径。如果以整个方法为单位进行编译，则不可避免地会在冷路径的编译上消耗时间和空间，但通过该路径获得执行加速的频率却非常低。因此基于 trace 的 JIT 编译会对方法作进一步划分，根据方法内具体分支的执行频率，只对热路径进行编译；而在生成代码的冷路径入口处插入冷路径陷阱（uncommon trap），使得进入冷路径时可以返回解释器采用解释执行[36]。可以发现，基于 trace 的 JIT 的显著优点在于能有效捕获循环体中的热路径。

在上文引述的 Google 的相关报告中曾测试，在一个大小约为 4.5MB 的测试程序中，热点方法占到整个程序字节数的 8%，而热点方法中热点路径的占比仅为 26%。因此基于 trace 的 JIT 相比于基于 method 的 JIT 具有更高的“程序加速比/编译消耗”

比值。

在实际运行中，Android 系统会根据系统状态（如是否接入外接电源），将 Dalvik JIT 编译器设定为 method 方式或 trace 方式。而两种方式下 Dalvik 虚拟机编译执行的工作流程大致相同，如图 5-1 所示。

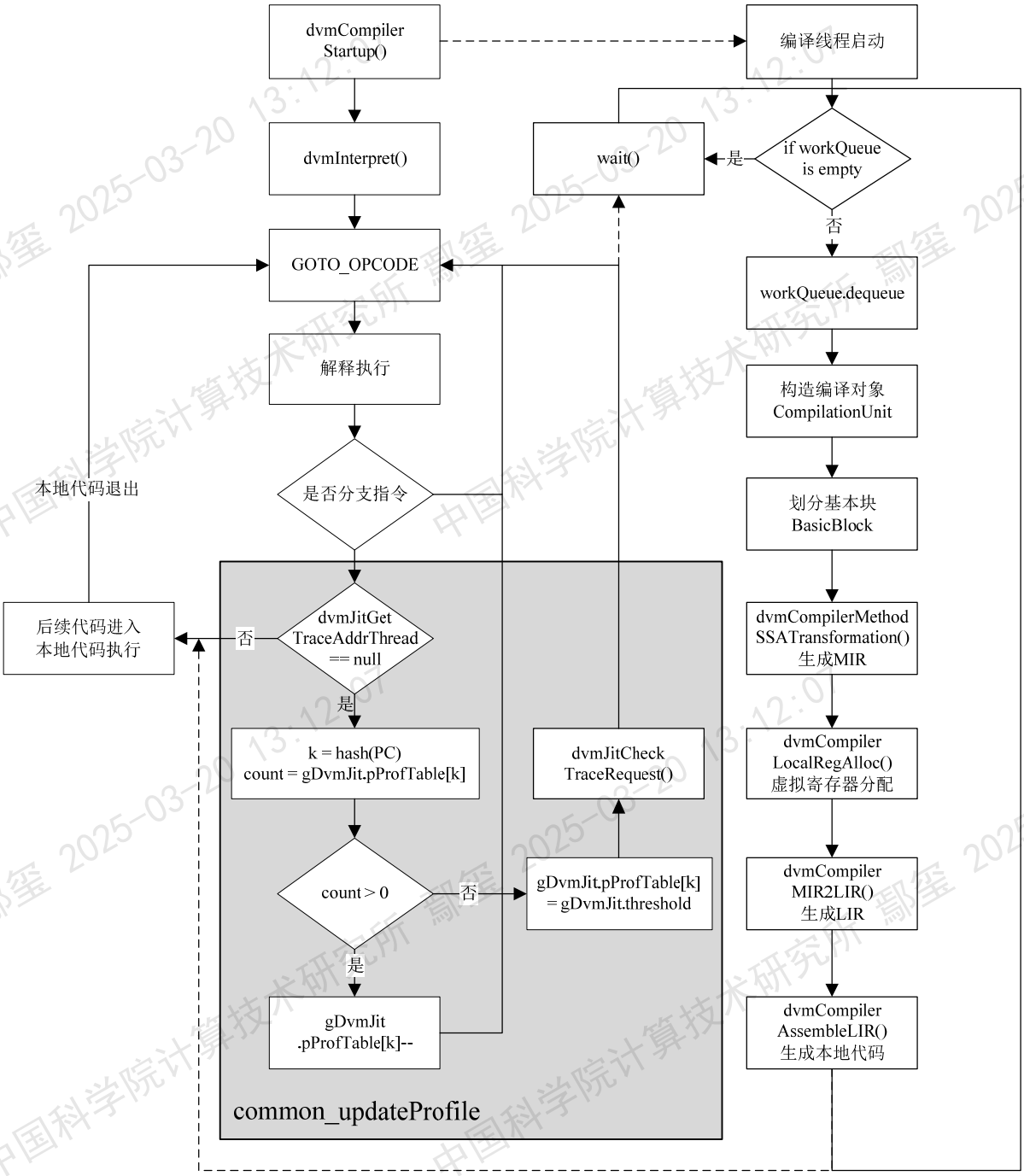

图 5-1 Dalvik JIT 编译模式工作流程

首先对上图中涉及的全局变量和数据结构作简单解释：

（1）Dalvik 虚拟机中使用的全局变量

- struct DvmGlobals gDvm，保存虚拟机运行状态的全局变量。
- struct DvmJitGlobals gDvmJit，保存 JIT 编译器运行状态的全局变量。

（2）JIT 模块中使用的关键数据结构

- dex 代码，即 dex 格式的 Dalvik 字节码，作为 JIT 编译器的输入。
- CompilationUnit，编译单元数据结构，用以保存一个被编译对象即一个 method 或 trace 所有相关的编译信息。
- BasicBlock，基本块数据结构，基本块是描述程序控制流的最小单元，它具有唯一的入口点和出口点，程序的控制流图（Control Flow Graph）可以表示为一些列基本块的连接。大多数编译器在内部都采用控制流图和基本块对程序进行抽象表示，Dalvik JIT 编译器亦如此。
- MIR（Middle-level Intermediate Representation），在编译器内部使用的中级中间表示，Dalvik JIT 编译器中采用 SSA 形式（Static Single Assignment Form）来作为中级中间表示，即在编译过程中输入的 dex 代码将首先被转换为 SSA 形式。
- LIR（Low-level Intermediate Representation），在编译器内部使用的低级中间表示，在 Dalvik JIT 编译器中指对 MIR 作寄存器分配以及全局优化后生成的表示形式，转换为 LIR 表示的代码已经与目标平台的机器码基本等价。
- native code，本地代码，即可以在目标平台上直接运行的机器码。

基于上述的全局变量和数据结构，以下具体分析流程图中各环节的工作：

（1） Dalvik 虚拟机启动后，主线程会调用 dvmCompilerStartup 函数创建编译线程，然后进入 dvmInterpret 函数即解释器入口开始解释执行；编译线程启动后会进行 JIT 编译器的初始化，然后在 workQueue 即编译任务队列上等待。

（2） 一般情况下，解释器通过 GOTO_OPCODE 操作对字节码进行逐条解释执行，当遇到 L_OP_PACKED_SWITCH、L_OP_IF_EQ 等 14 条带有分支功能字节码指令后，解释器将执行 common_updateProfile 附加操作。

（3） 在 common_updateProfile 操作中，解释器将首先调用 dvmJitGetTraceAddr-Thread 函数试图获取已编译代码的存储地址。如果 dvmJitGetTraceAddrThread 返回值不为 null，说明该条字节码所在的 method 或 trace 已经被编译为本地代码，则直接从解释器跳转到该地址开始本地执行；如果返回值为 null，则说明该段字节码尚未经过编译，于是对该条字节码在 Dalvik 内部的程序计数器记录也即 PC 值作 hash 运算 k = hash(PC)，并用 hash 值查表获得 count = gDvmJit.pProfTable[k]，count 即为对该条字节码启用编译执行的剩余计数，根据 count 值作如下判断：

A. 如果 count > 0，则令 gDvmJit.pProfTable[k]--，然后解释执行下一条字节码；

B. 如果 count = 0，则令 gDvmJit.pProfTable[k] = gDvmJit.threshold，并调用 dvmJitCheckTraceRequest 向 workQueue 队列中添加编译任务，然后再继续解释执行下一条字节码。

（4） 编译线程从 workQueue 中取出编译任务，调用编译器进行编译，编译器的工作流程如下：

A. 基于输入的 dex 代码，创建 method 或 trace 所对应的 CompilationUnit 对象；

B. 基于 CompilationUnit 对象对 method 或 trace 进行解析，将其划分为若干个 BasicBlock，以这些 BasicBlock 为结点构建 Control Flow Graph；

C. 调用 dvmCompilerMethodSSATransformation，为 CompilationUnit 中所有的 BasicBlock 添加对应的 MIR 表示；

D. 调用 dvmCompilerLocalRegAlloc，对 MIR 表示中使用到的 SSA 虚拟寄存器进行全局编号，并推断寄存器类型

E. 调用 dvmCompilerMIR2LIR 将 MIR 转换为对应的 LIR 表示，在此过程中会为虚拟寄存器分配实际的物理寄存器；

F. 遍历 LIR，调用 dvmCompilerAssembleLIR 将 LIR 转换为机器码；在遍历完成后，CompilationUnit 即被编译为目标平台本地代码，将这些代码保存到“/data/dalvik-cache/”目录下的相应文件中，供解释器调用。

（5） 当再次执行到该 PC 处时，即会在步骤（3）所述的 common_updateProfile 操作中通过 dvmJitGetTraceAddrThread 函数从“/data/dalvik-cache/”目录下获得编译生成的本地代码的位置，然后在后续代码中转为本地执行，直至从本地代码返回至解释器。

从 Dalvik JIT 编译器工作流程中可以看出，dvmCompilerMIR2LIR、dvmCompiler-AssembleLIR 等低级表示转换函数的实现均与目标平台体的系结构相关，目前 Dalvik JIT 编译器已经实现了对 ARM、x86、MIPS 三种平台的兼容。但由于龙芯 GS464 核心在寄存器分配规则上与标准的 MIPS 体系结构间存在差异，导致 Dalvik JIT 编译器针对 MIPS 平台的实现并不兼容龙芯 3A 平台，因此无法在龙芯 3A 平台上直接使用 Dalvik JIT 优化技术。

针对这种情况，本课题首先实现了 Dalvik JIT 编译器向龙芯 3A 平台的移植，从而在龙芯 3A 平台上成功实现了 JIT 优化技术；此外，本课题还提出并实现了基于复合评价模型的热度检测算法，该算法可以实现对 Dalvik JIT 技术的进一步优化，且不具有平台相关性可适用于各类搭载 Dalvik JIT 技术的硬件平台。

5.3 基于龙芯 3A 的 Dalvik JIT 编译器移植

在第三章阐述 Android 系统移植工作时已说明，龙芯 3A 处理器原生兼容 MIPS64 指令集，但由于 Android 操作系统目前只支持 32 位模式，因此移植后的系统在运行时利用了龙芯 3A 对 MIPS32 指令集的向下兼容性。根据 MIPS64 指令集的设计规范，当处理器的 Status 控制寄存器的 FR 位置为 1 时，处理器的浮点寄存器宽度将被设定为 64 位，可用的浮点寄存器个数为 32 个；而当 Status 控制寄存器的 FR 位置为 0 时，浮点寄存器宽度将被设定为 32 位，可用的浮点寄存器个数为 32 个，此时才具有对 MIP32 规范的向下兼容性[37]。由此可见，为了支持 32 位操作系统，必须将 Status 控制寄存器的 FR 位置为 0。

但龙芯 3A 处理器的设计与上述 MIPS64 规范有所不同：当 Status 控制寄存器的 FR 位置为 1 时，龙芯 3A 处理器的状态与 MIPS64 规范的描述完全一致；当 Status 控制寄存器的 FR 位置为 0 时，龙芯 3A 处理器的浮点寄存器宽度亦为 32 位，但可用的浮点寄存器个数只有 16 个，也即在 0 号~31 号这 32 个浮点寄存器中只有 0，2，...，30 这 16 个偶数号寄存器可用，而任何用到奇数号浮点寄存器的指令将无法返回预计的结果。

在上节分析 Dalvik JIT 编译器的工作流程时曾介绍，在 MIR 向 LIR 转换的过程中会为 SSA 虚拟寄存器分配实际的物理寄存器；而原版 JIT 编译器的物理寄存器分配规则完全遵照 MIPS32 规范，即默认可以使用 32 个浮点寄存器。因此，向龙芯 3A 平台移植 Dalvik JIT 编译器的核心工作，即是重构 MIR-LIR 转换过程中的浮点寄存器分配规则。

在原有的寄存器分配调用流程中，实际实现物理寄存器分配的是 allocTempBody 函数，该函数被调用时，会遍历指定的寄存器池，从中查找到编号最小的空闲寄存器返回给调用者。根据 MIPS32 规范，32 位通用寄存器与 32 位浮点寄存器的个数均为 32 个，因此在寄存器分配模块中浮点寄存器和通用寄存器的分配规则完全相同，只需重用 allocTempBody 函数即可。

而为了兼容龙芯 3A 平台浮点寄存器分配，在移植工作中需要对分别实现浮点寄存器分配与通用寄存器分配功能。在通用寄存器分配时，以 1 为步进遍历通用寄存器池中的全部 32 个寄存器，分配其中的可用者；而在浮点寄存器分配时，以 2 为步进遍历浮点寄存器池中的 0，2，…，30 号寄存器，分配这 16 个寄存器中的可用者，从而避免对奇数号浮点寄存器进行分配。如图 5-3 所示是经过重构后的寄存器分配模块的架构。

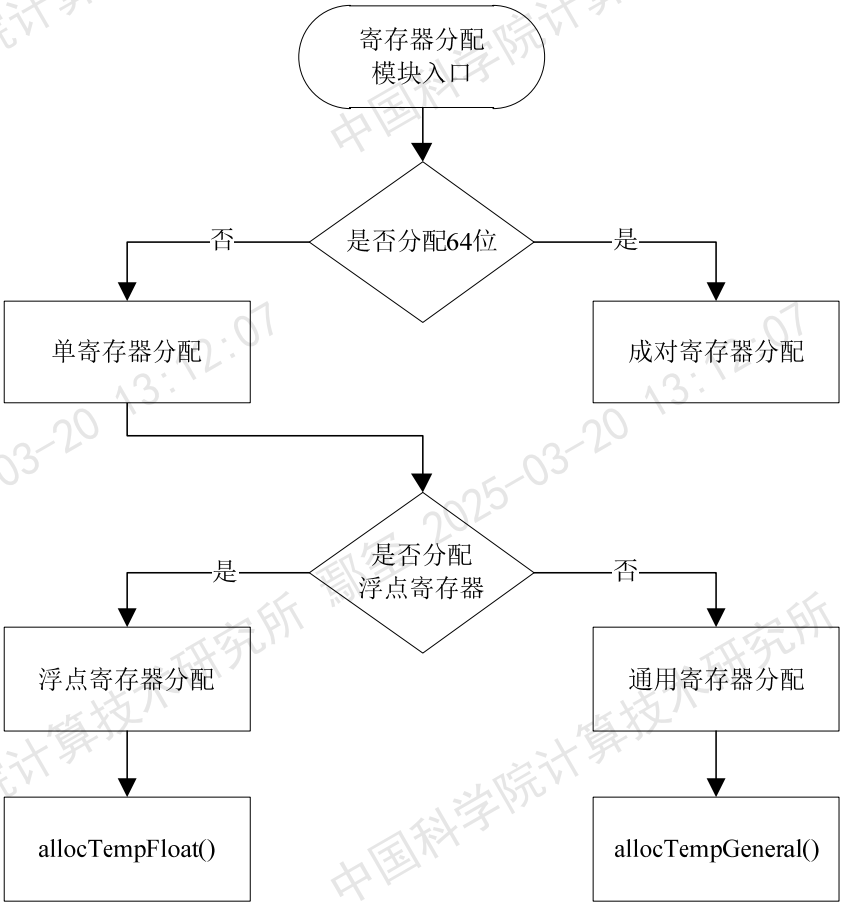

图 5-2 针对龙芯 3A 的寄存器分配模块架构

其中 allocTempFloat 浮点寄存器分配函数的分配算法伪代码描述如下。

代码 5-1 浮点寄存器分配算法的伪代码描述

```
int allocTempFloat(RegisterInfo *p) {
    for ( i = 0; i <= 31; i = i +2) {
        查看浮点寄存器池 p 中的寄存器 p[i]是否被占用;
        if 寄存器 p[i]可用 {
            更新寄存器 p[i]的状态信息;
            return i;
    }
    return -1;
}
```

经过以上重构后，即可以实现在龙芯 3A 平台上启用 Dalvik JIT 优化技术。在 5.5 节中将对采用 Dalvik JIT 优化后带来的性能加速比作进一步对比测试及分析。下面将阐述本课题基于 JIT 技术实现的热度评价模型与热度检测算法优化。

5.4 基于复合评价模型的热度检测算法优化

当 Java 程序被加载至 Dalvik 虚拟机进行执行时，虚拟机主线程会创建一个独立的执行线程与该 Java 程序绑定，执行线程负责对与之绑定的 Java 程序进行执行以及热度检测；在 Java 程序退出后，其对应的执行线程将被父线程回收。如图 5-4 是执行线程在解释执行与热度检测阶段的工作流程。

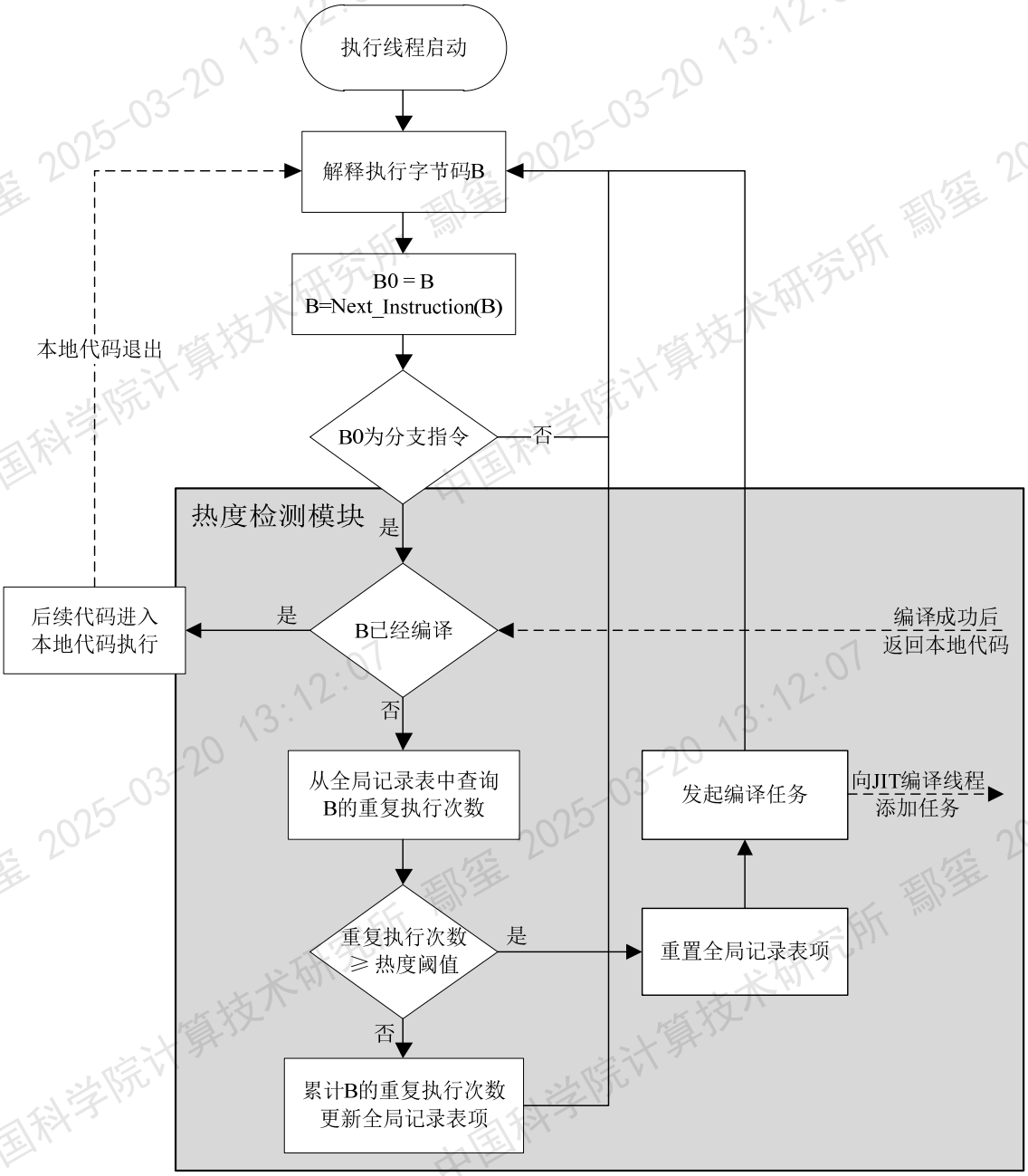

图 5-3 Dalvik 中的代码热度检测流程

由于 JIT 即时编译只选择性地对“高热度”代码进行编译，因此在开始编译之前

需要消耗一定的时间用于收集程序执行信息，从而根据信息进行代码热度检测；而在这段时间中该段代码依旧采用效率较低的解释执行，所以这一阶段也称为 JIT 冷启动阶段[38]。根据上图所示的工作流程可以归纳出：对于该 Java 程序中的某条字节码 B，其在 JIT 冷启动阶段的持续时间 T_{cold} 可以近似为如下数学表示：

T_{begin}：　字节码 B 第一次被解释执行的时刻；

T_{end}：　虚拟机判定 B 为热代码，并触发编译的时刻；

t[i]：　字节码 B 从第 i - 1 次解释执行，到第 i 次解释执行之间的时间间隔，（i≥2），t[1] = 0；

$$T_{cold} = T_{end} - T_{begin}$$
$$= t[2] + t[3] + \cdots + t[Threshold] = \sum_{i=1}^{Threshold} t[i] \quad (5\text{-}2)$$

表达式 5-2 中的 Threshold 参数表示执行线程在执行热度检测时所采用热度阈值，也即当代码的重复执行次数达到 Threshold 时将触发编译。在虚拟机主线程创建执行线程时，会根据系统参数对执行线程的热度阈值进行设定；当执行线程在执行过程中检测到某条字节码被解释执行的次数、也即该代码的热度达到热度阈值时，会将其提交至 JIT 编译线程进行编译。因此，在执行线程的整个生命周期中，其所采用的热度阈值为一个常数。

可以看出，以上热度检测流程是根据简化程度极高的程序热度评价模型进行设计的，该模型采用代码的重复执行次数作为评价其热度的唯一指标；且对于程序中的任意执行上下文，均简单地采用一个恒定的热度阈值判定其是否应当被编译。该热度模型的优点在于其复杂低且实现简单；而不足则在于其忽略了程序的执行上下文的时间特性与空间特性，因此也无法利用执行时间、执行位置等执行开始后所产生的信息来综合评价代码热度，进而更准确地筛选出具有编译价值的代码。

时间局部性是程序的重要特性之一，例如在上述的执行线程中，在近期内被频繁执行的某段代码，将会有一定概率在未来一段时间内仍被频繁执行，也即某段代码的执行频率这一统计特性对于其编译价值的判定具有重要的参考意义[39]。根据这一假设，本课题在上述热度模型的基础上，通过引入代码的重复执行频率也即重复执行间隔这一指标，构建了基于复合评价的热度模型。在该模型中，定义代码的热度分值 H 为代码的重复执行次数 C 与重复执行间隔 I 的符合函数 f，即有如下数学表示：

$$H = f(C, I) \quad (5\text{-}3)$$

该模型对应的热度检测过程，即可抽象为已知程序的运行信息 C 与 I、根据打分

函数 f 对代码的热度进行打分，并将打分分值大于热度阈值 Threshold 的代码提交编译的过程。其具体流程如图 5-5 所示。

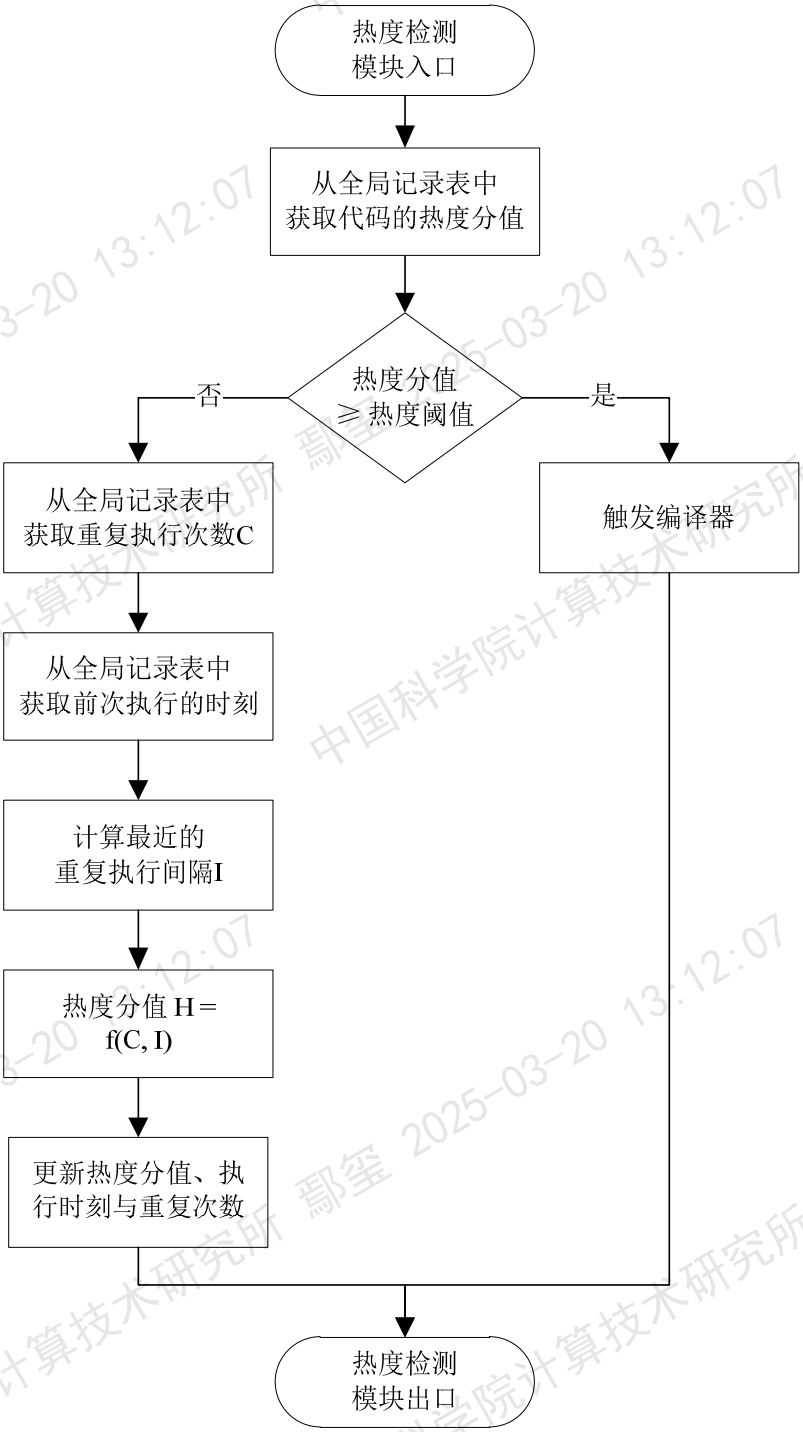

图 5-4 基于复合评价模型的热度检测算法流程

可以看出，基于重复执行次数的热度模型可以视为复合评价模型在 $f(C,I)=C$ 时的一个特例。在复合评价模型的具体实现过程中，考虑到实现复杂度与内存占用量等因素，对热度检测算法进行了进一步简化，如在热度检测模块中设计了该模块的全

局调用次数计数器，在执行线程对某代码进行热度检测时，即以全局调用次数计数器的计数值作为当前时刻的相对表示；并且采用累计加分的方法实现打分函数f。以下伪代码描述了采用上述热度检测算法后的执行线程的工作流程。

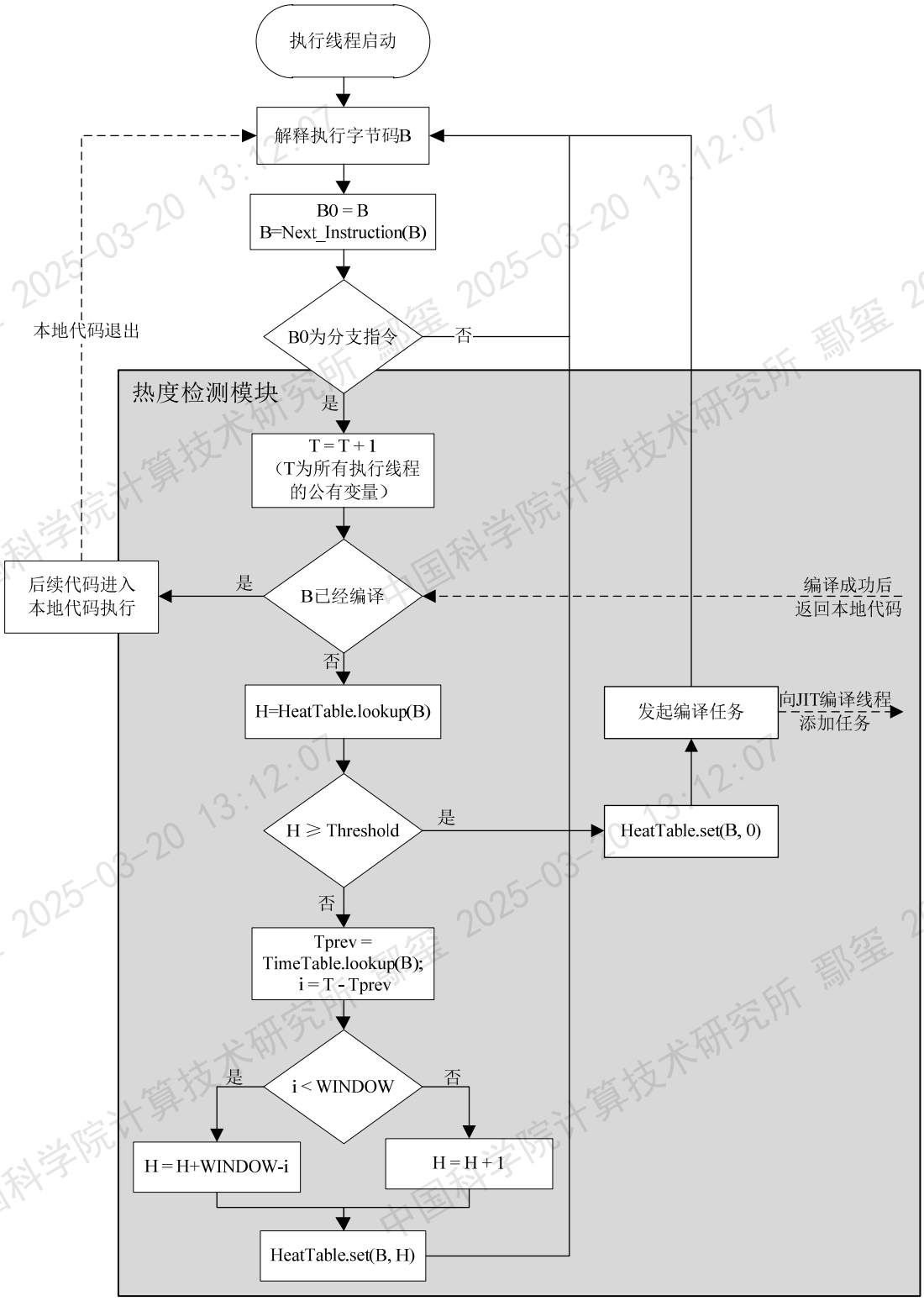

图 5-5 基于复合评价模型的热度检测算法在 Dalvik 中的实现

从以上执行流程可以看出，基于复合评价模型的热度检测算法可以理解为，在执行过程中，根据代码的重复执行次数、代码的最近重复执行间隔两向指标对代码热度进行累计加分，当热度分值累计超过热度阈值时将触发编译。因此在该检测算法下，冷启动阶段的持续时间 T_{cold} 可以近似为如下数学表示：

$$T_{cold} = \sum_{i=1}^{Threshold-E} t[i] \tag{5-4}$$

其中 E 表示重复执行间隔指标对代码热度分值的贡献。由式 5-4 可知，在同一执行线程所执行的代码中，执行间隔较短也即执行频率较高的代码，只需要较少的重复执行次数即可触发编译，也即缩短了其对应的 Tcold 时间；因此执行线程可以更早地进入效率较高的编译执行方式，使整体执行效率得以提高。采用基于复合评价模型的热度检测算法后相比于原有模型带来的性能提升，将在 5.5 节中进行具体的对比测试评估。

5.5 优化效果的对比测试与评估

为评估以上两种优化方法对 Android Java 性能的实际提升效果，本课题采用 AndEBench、Linpack、Scimark2、Benchmark Pi 四个 Android 平台上较为常用的性能测试工具，分别对优化前的解释执行模式、启用移植后的 JIT 编译器的 JIT 模式（下简称优化方法一）、以及在优化方法一的基础上采用基于复合评价模型的热度检测算法优化后的 JIT 模式（下简称优化方法二）进行了性能测试。其中各测试集在采用优化方法一后，与解释执行模式的性能得分比，也即性能加速比如表 5-1 所示。

表 5-1 优化方法一与解释执行模式的性能对比

	AndEBench	Linpack	Scimark2	BenchmarkPi
性能加速比/倍	2.19	2.65	4.24	4.19

*优化前与优化后的性能得分均为 3 次测试的算术平均值

从表 5-1 中可以看出，在成功移植 Dalvik JIT 编译器并启用 JIT 模式后，各测试集得到的性能评分可达到原先解释执行模式的 2 至 4 倍，因此优化方法一对比纯解释模式的性能提升效果非常明显。如表 5-2 所示为采用优化方法二后，在优化方法一的基础上进一步获得的性能得分提升百分比。图 5-6 为以解释执行模式得分为基数归一化后的优化方法一与优化方法二的性能对比。

表 5-2 优化方法二与优化方法一的性能对比

	AndEBench	Linpack	Scimark2	BenchmarkPi
性能提升/%	2.28	7.54	5.42	8.11

*优化前与优化后的性能得分均为 3 次测试的算术平均值；优化方法二中 Threshold = 200；Window = 15

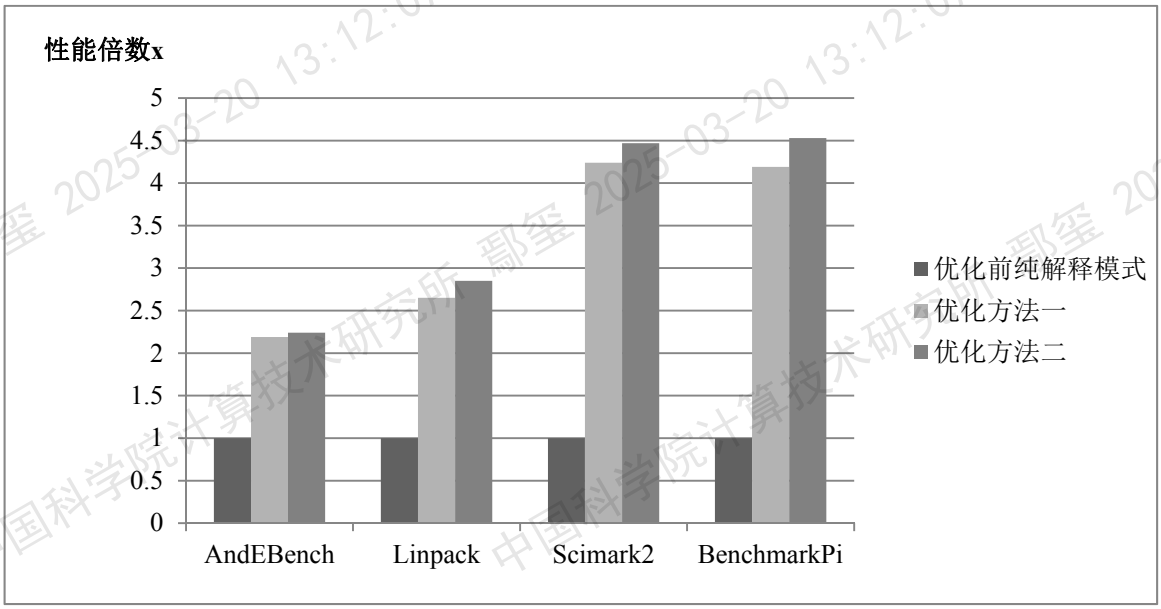

图 5-6 两种优化方法归一化后的性能对比

从表 5-2 中可以看出，在启用 JIT 编译的基础上，采用基于复合评价模型的热度检测算法优化后，可以进一步获得 2%至 8%的性能提升。具体分析以上测试集的测试原理可以发现，由于在测试算法的实现中使用了较多的高频循环，因此采用基于复合评价模型的热度检测算法后可以在更少的循环次数内检测出其热代码属性，也即减少了解释执行次数占循环执行总次数的比例，因而提高了整个测试算法执行过程中的执行效率。

5.6 本章小结

本章首先通过构建并分析 Android 应用性能模型，指出了 Dalvik 虚拟机执行效率优化对于 Android 应用性能提升的关键作用，进而详细分析了 JIT 即时编译技术这一优化虚拟机执行效率的关键技术在 Dalvik 虚拟机中的工作原理；然后通过对 Dalvik JIT 编译器寄存器分配模块的重构实现了 JIT 编译器对龙芯 3A 的兼容，从而在龙芯 3A 平台上成功应用了 Dalvik JIT 优化方法；经过多款测试工具的实际评估，该方法可产生 2 至 4 倍的性能加速比；在此基础上，本章进一步创造性地提出并实现了适用于

任意平台的基于复合评价模型的热度检测算法；经过实际测试发现，通过该算法的优化可以 JIT 优化方法的基础上进一步获得 5%至 8%的性能提升。

中国科学院计算技术研究所 鄢銮 2025-03-20 13:12:07

第六章 总结与展望

本文在深入分析 Android 系统架构及关键组件功能，并对系统移植类问题的解决方法论进行归纳总结的基础上，围绕基于龙芯 3A 平台的 Android 系统移植与 Dalvik 虚拟机优化两项研究主题进行了全面阐述，其中的关键工作与主要创新点可总结为以下几个方面：

（一）通过对大量系统移植类问题进行总结分析，将该类问题的解决方法归纳为开发环境的搭建、系统源码的重构与二次开发、源码编译以及目标程序部署调试，移植成果测试、系统性能优化、优化效果的对比评估这六个关键环节，并以上述方法论为指导展开本文的具体工作。

（二）以龙芯 3A780E 参考系统为目标平台，通过对 Android Linux 内核及 Android 文件系统的相关模块进行二次开发与重构，实现了 Android 4.0.4 操作系统在目标平台上的成功运行；并在此基础上完成了移植后系统的测试评估，验证了龙芯 3A-Android 系统已具备基本实用性。

（三）通过 Android 应用性能模型的构建及分析，确定了 Dalvik 虚拟机执行效率优化这一 Android 系统优化的关键方向；进而通过对 Dalvik JIT 编译器中的寄存器分配模块进行重构，实现了 Dalvik JIT 优化技术在龙芯 3A-Android 系统上的成功应用，并采用相关测试集验证了该优化方法可实现 2 至 4 倍的性能加速比。

（四）在 Dalvik JIT 原有的代码热度检测算法基础上，创新性地提出并实现了基于复合评价模型的热度检测算法；根据测试集的评估结果，采用该算法后 JIT 优化方法可进一步获得 5%至 8%的性能提升；由于该算法不具有平台相关性，亦可适用于龙芯平台以外的其他硬件平台。

基于上述研究成果，结合龙芯处理器平台以及 Android 操作系统的发展方向，未来可以在以下研究方向作进一步探索：

（一）通过静态二进制翻译或动态二进制翻译技术，进一步提高龙芯平台上 Android 系统的应用兼容性。在 4.3 节中已分析，由于部分 Android 应用中使用了基于 ARM 平台的本地代码，导致其无法在龙芯 3A-Android 系统中正常运行。未来可以结合 Dalvik JNI 的实现规则采用静态二进制翻译技术，在 Android 应用安装或初次运行时将其中的 ARM 平台本地代码翻译为龙芯平台本地代码；或者在应用运行时采用动态二进制翻译技术对 ARM 本地代码进行实时翻译。

（二）进一步深入研究于复合热度评价模型中各模型参数对代码热度检测效率的影响。在不同的应用场景中，程序中代码的先验热度特征将有所区别，通过对应用场

景的统计分析，并根据统计模型对复合热度评价模型中的评价函数进行针对性的优化，有望能进一步提升虚拟机的代码热度检测效率。

中国科学院计算技术研究所 鄢銮 2025-03-20 13:12:07

参考文献

[1] Google Inc. What is Android[EB/OL]. [2012]. http://developer.android.com/guide/basics/what-is-android.html.

[2] Open Handset Alliance. Android Overview[EB/OL]. [2012]. http://www.openhandsetalliance.com/android_overview.html.

[3] Android Open Source Project. Philosophy and Goals[EB/OL]. [2012]. http://source.android.com/about/philosophy.html.

[4] Google Inc. Discover Android[EB/OL]. [2012]. http://www.android.com/about/.

[5] Hugo Barra. Android Keynote[EB/OL]. [2012]. https://developers.google.com/events/io/2012/.

[6] Bornstein D. Dalvik VM Internals[C]. Google I/O Developer Conference. 2008

[7] ARM Holdings. Getting Started with Android[EB/OL]. [2012]. http://www.arm.com/community/software-enablement/google/solution-center-android/getting-started-with-android-on-arm.php.

[8] 龙芯中科技术有限公司. 龙芯 3A 处理器用户手册[EB/OL]. [2012]. http://www.loongson.cn/.

[9] Xiang Gao,Yun-Ji Chen,Huan-Dong Wang,Dan Tang,Wei-Wu Hu. System Architecture of Godson-3 Multi-Core Processors[J]. Journal of Computer Science and Technology, 2010(25):181-191.

[10] Wikipedia. Android (Operating System)[EB/OL]. [2012]. http://en.wikipedia.org/wiki/Android_(operating_system).

[11] MIPS Technologies. Getting Started with Android[EB/OL]. [2012]. http://developer.mips.com/android/getting-started-with-android/

[12] 雷鸣,靳婷,徐晓亮,齐锦. 面向龙芯平台的 Android 系统移植研究[J]. 计算机工程与应用. 2012(22)

[13] 董剑,张源,杨珉.利用硬件加速层优化 Android 显示系统[J]. 小型微型计算机系统. 2012(07)

[14] 叶炳发,孟小华. Android 图形系统的分析与移植[J]. 电信科学. 2010(02)

[15] 吴少刚,邹国民. Dalvik 虚拟机在龙芯平台上的移植与实现[J]. 计算机工程. 2011(22)

[16] 姚昱旻,刘卫国. Android 的架构与应用开发研究[J]. 计算机系统应用. 2008(11)

[17] 罗升阳著. Android 系统源代码情景分析[M]. 电子工业出版社. 2012

[18] 柯元旦著. Android 内核剖析[M]. 电子工业出版社. 2011

[19] Android Open Source Project. Dalvik Executable Format[EB/OL]. [2007]. http://source.android.com/tech/dalvik/dex-format.html. 2007

[20] Yunhe Shi,David Gregg,Andrew Beatty. Virtual Machine Showdown: Stack versus Registers[C].

ACM Transactions on Architecture and Code Optimization. 2005

[21] Hyeong-Seok O,Beom-Jun K,Hyung-Kyu C,Soo-Mook M. Evaluation of Android Dalvik virtual machine[J]. JTRES '12 Proceedings of the 10th International Workshop on Java Technologies for Real-time and Embedded Systems. 2012: 115-124

[22] Android Open Source Project. Dalvik VM Instruction Formats[EB/OL]. http://source.android.com/tech/dalvik/instruction-formats.html. 2007

[23] Android Open Source Project. Bytecode for the Dalvik VM[EB/OL]. http://source.android.com/tech/dalvik/dalvik-bytecode.html. 2007

[24] (韩)金泰延,宋亨周等著,武传海译. Android 框架揭秘[M]. 人民邮电出版社. 2012

[25] 林仕鼎,任爱华,王雷,周伯生. Linux 内核在新型硬件平台上的实现[J]. 北京航空航天大学学报. 2003(03)

[26] (美)Ron Patton 著,张小松等译. 软件测试:第二版[M]. 机械工业出版社. 2006

[27] 毛德操,胡希明著. Linux 内核源代码情景分析[M]. 浙江大学出版社, 2001

[28] 孟小甫,高翔,从明,张爽爽. 龙芯 3A 多核处理器系统级性能优化与分析[J]. 计算机研究与发展. 2012(S1)

[29] 邓凡平著. 深入理解 Android:卷 I[M]. 机械工业出版社. 2011

[30] Android-x86 Project. Porting Android to x86[EB/OL]. [2012]. http://www.android-x86.org/

[31] The Mesa 3D Graphics Library. Platforms and Drivers[EB/OL]. [2012]. http://www.mesa3d.org/systems.html

[32] Bin Zhou. Xbench: A benchmark evaluating the performance of graphic display system in virtual machines[C]. Proc. 5th International Conference on Computer Science and Education (ICCSE). 2010: 1730-1734

[33] Adl-Tabatabai A,Cieniak M,Lueh GY,et al. Fast, Effective Code Generation in a Just-In-Time Java Compiler[C]. Proceedings of the ACM SIGPLAN'98 Conference on Programming Language Design and Implementation. 1998

[34] Ben Cheng,Bill Buzbee. A JIT Compiler for Android Dalvik VM[C]. Google I/O Developer Conference. 2010

[35] Paleczny M,Vick C,Click C. The Java hotspot server compiler[C]. Proc.of the Java Virtual Machine Research and Technology Symp. 2001

[36] Andreas Gal,Christian W.Probst,Michael Franz. Hotpath VM: An Effective JIT Compiler for Recource-constrained Devices[C]. International Conference on Virtual Execution Environments. 2006

[37] (英)Dominic Sweetman 著,赵俊良,张福新,陶品译. MIPS 处理器设计透视[M]. 北京航空航天大学出版社, 2005

[38] Seong-Won L,Soo-Mook M,Seong-Moo K. Enhanced Hot Spot Detection Heuristics for Embedded Java Just-in-Time Compilers[C]. Proc. of the 2008 ACM SIGPLAN-SIGBED conference on Languages, compilers, and tools for embedded systems. 2008

[39] (美)Bill Venners 著,曹晓钢,蒋靖译. 深入 Java 虚拟机[M]. 机械工业出版社, 2003

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07

致 谢

三年的研究生求学生涯就要结束了，在临别之际我首先要感谢自己的导师胡伟武研究员。在这三年中，胡老师不仅在生活上给予我们这批学生以支持，使我们能够免除后顾之忧全身心投入科研工作，同时胡老师的言传身教更深深地影响了我们年轻人做人做事业的态度，使我们体会到一名心系家国的科研工作者应有的抱负与担当。跟随胡老师学习的这段经历将成为我成长道路上的宝贵财富。

感谢王剑老师，章隆兵老师和沈海华老师这三年来给予我们这一届学生的帮助，在几位老师的引导下，我们得以在广阔的科研天地中摸索到适合自己的角色和位置，并明确了自己今后的发展方向。

感谢办公室的杨杰老师和刘凤芹老师的照顾，求学期间有诸多琐事都依靠两位老师的耐心帮助才得以顺利解决。

特别感谢苏孟豪博士在这三年中对我在技术上和工作方法上的指导。三年前我刚进入实验室时是一个彻头彻尾的菜鸟，而毕业时我已经能够深入一些技术领域甚至在一些技术难点上有所作为，这其间有赖于苏孟豪的指点让我迅速发现并弥补自己的知识盲点，在与他的不断沟通中我获得了长足的进步。

感谢高翔博士和靳国杰博士在本文的技术路线选择等关键问题上给予我的指导，感谢王一光同学和蓝小明同学在本文的实验环境搭建方面给予我的帮助。

感谢相伴度过三年研究生生涯的同学和同事们——薛原，马君，胡立，易龙，文杰，李铭夫，黎进，薛双百，晏杨，许昊宇，刘凡超，王静文，王喆，史鑫磊，朱健，陆超，王磊，刘道福，伍岳，李鹏，陈华军，段玮，尹一笑，刘苏，张尧，姜文奇，吴龙，黄帅，符兴建……有幸能与他们相识是这三年给予我的珍贵礼物。

最后我要感谢我的父亲和母亲，是他们从无条件、从不停歇的关怀和支持，二十六年来一路护送着我直到今天。感谢我的未婚妻，这七年中她是我成长的首席助推者，也是我每个重要决定的第一个支持者，谨以此文致我们即将逝去的青春。

孙浩志

2013 年 5 月

作者简历

【个人简历】

姓名：孙浩志　　性别：男　　出生日期：1987.6.15　　籍贯：山西太原

【学习经历】

2010.9～2013.6　　中国科学院计算技术研究所　计算机系统结构专业　硕士研究生

2006.9～2010.7　　浙江大学　通信工程专业　本科

【攻读硕士学位期间参加科研项目】

国家“核高基”科技重大专项项目，高性能多核 CPU 研发与应用（2009ZX01028-002-003），2009－2011

中国科学院计算技术研究所 鄢玺 2025-03-20 13:12:07